

UAV–Digital Twin–AI Intrusion Detection Framework
Technical Implementation Manual (Expanded Edition)
From Clean Ubuntu Workstation to Live Cyber Defence

UAV Cyber-Range Laboratory
Project repository: `uav-cyber-ml`

July 29, 2026

Expanded handbook edition: written for beginners and specialists. Every major concept is introduced before use. Screenshot placeholders mark where to insert real Physical Twin and Digital Twin captures from your laboratory.

Contents

About This Manual	xi
I Phase 1 — Building the UAV Environment	1
1 System Requirements	2
1.1 Purpose	2
1.2 Physical Twin (UAV PC)	2
1.3 Digital Twin / IDS host	3
1.4 OS versions	3
1.5 Verification	3
2 Required Software	4
2.1 Master software inventory	4
2.2 Where each package runs	5
2.3 Python dependency pin file	5
2.4 Verify imports	5
3 Linux Preparation	6
3.1 Update Ubuntu	6
3.2 Build tools	6
3.3 Enable SSH server	6
3.4 PATH	7
3.5 Verification	7
3.6 Troubleshooting	7
4 Linux Networking for the Cyber-Range	8
4.1 Why networking matters here	8
4.2 IP addresses	8
4.3 localhost	8
4.4 Network interfaces	8
4.5 Ethernet vs Wi-Fi	9
4.6 Static vs dynamic IP	9

4.7	Routing	9
4.8	Ports	9
4.9	Firewalls	9
4.10	ping	10
4.11	SSH and SCP	10
4.12	ROS 2 / DDS networking (optional path)	10
4.13	MAVLink networking	10
4.14	Troubleshooting and verification	11
5	Installing ROS 2	12
5.1	Before installation — what is ROS 2?	12
5.1.1	Role in a UAV lab	12
5.1.2	How ROS 2 relates to other components	12
5.1.3	Supported versions	12
5.2	Installation (Humble on Ubuntu 22.04)	12
5.3	Environment setup	13
5.4	Verification and testing	13
5.5	Uninstall (optional)	14
5.6	Troubleshooting	14
6	Installing Gazebo	15
6.1	Before installation	15
6.1.1	Dependencies and compatibility	15
6.2	Installation via PX4 dependency script	15
6.3	Verification	16
6.4	Common failures	16
6.5	Uninstall notes	16
7	Installing PX4	17
7.1	Before installation — PX4 concepts	17
7.2	Clone and build	17
7.3	Lab remote start	18
7.4	Verification	18
7.5	Troubleshooting	18
8	Communication Protocols in Depth	19
8.1	MAVLink from first principles	19
8.2	Framing	19
8.3	Lab ports	19
8.4	Trusted vs attacker traffic	19
8.5	pymavlink	20
8.6	DDS / Micro XRCE-DDS / Fast DDS	20
8.7	MAVSDK and MAVROS	20
8.8	Troubleshooting	20

9	Running the First UAV Simulation	21
9.1	Goals	21
9.2	On the UAV PC (local first flight)	21
9.3	Optional QGroundControl	22
9.4	From the Digital Twin host	22
9.5	Landing and shutdown	23
9.6	Troubleshooting	23
10	Phase 1 Troubleshooting	24
10.1	Symptom matrix	24
10.2	Network and SSH checklist	25
10.3	Display and Gazebo	25
10.4	PX4 build failures	25
10.5	Port conflicts	25
10.6	ROS 2 optional path	26
11	Phase 1 Summary	27
11.1	You should now be able to	27
11.2	Exit checklist	27
II	Phase 2 — Digital Twin Development	28
12	Digital Twin Overview	29
12.1	Lab topology	29
12.2	What the DT shows	29
12.3	Advantages	30
12.4	Limitations	30
12.5	Relationship to ROS 2	30
13	Digital Twin Host Hardware	31
13.1	Role of the DT host	31
13.2	Recommended hardware	31
13.3	Network attachment	31
13.4	Privileges	32
14	Software Installation on the Digital Twin Host	33
14.1	Before installation	33
14.2	Install	33
14.3	Network capture privilege	33
14.4	Launch verification	33
14.5	Troubleshooting	34
15	Connecting Physical Twin and Digital Twin	35
15.1	Goal	35

15.2	Preconditions	35
15.3	Connect workflow	35
15.4	API checks	35
15.5	Data path reminder	36
15.6	Troubleshooting	36
16	Digital Twin Architecture	37
16.1	Block architecture	37
16.2	Module map	37
16.3	Control vs telemetry planes	38
16.4	API surface (selected)	38
17	Data Collection	40
17.1	Physical layer	40
17.2	Network layer	40
17.3	Labels	40
17.4	File layout per run	40
17.5	Physical columns (selected)	41
17.6	Network columns (selected)	41
17.7	How to collect a clean set	41
17.8	Troubleshooting collection	41
18	Data Generation	42
18.1	Shared mission protocol	42
18.2	Attack gates	42
18.3	Commands	42
18.4	Tier A vs Tier B	43
18.5	Reproducibility tips	43
19	Data Preprocessing	44
19.1	Pipeline stages	44
19.2	Columns dropped from models	44
19.3	CNN sequences	44
19.4	Live vs offline normalisation	45
19.5	Practical commands	45
20	Digital Twin Scripts	46
20.1	orchestrator.py	46
20.2	ssh_control.py	46
20.3	mav_common.py	46
20.4	config.py	46
20.5	build_dataset.py	46
20.6	run_dashboard.sh	46
20.7	Essential launch	47

20.8 IDS CLI entrypoints	47
20.9 Environment overrides (common)	47
21 Digital Twin Services (Detailed)	48
21.1 Real-time telemetry	48
21.2 Live mission monitoring	49
21.3 UAV state monitoring	49
21.4 Sensor synchronisation	49
21.5 Data collection module	50
21.6 Data storage module	50
21.7 Data labelling module	50
21.8 Data preprocessing module	50
21.9 AI prediction module	50
21.10Live IDS module	50
21.11Live defence module	51
21.12Dashboard module	51
21.13Logging module	51
21.14Replay / dataset explorer	51
21.15Alert module	51
21.16Event timeline	51
21.17Mission replay	51
21.18Performance analytics	52
22 Cyber-Attack Scenarios (Complete Playbooks)	53
22.1 Shared protocol	53
22.1.1 C	54
22.2 Benign	54
22.3 GPS spoofing	54
22.4 Disarm injection	55
22.5 Mode change LAND / RTL	55
22.6 Mission injection	56
22.7 Command flood DoS	56
22.8 RC override	56
22.9 Param injection	56
22.10Tier B playbooks	56
22.11Multi-attack tour	57
22.12Troubleshooting playbooks	57
23 Phase 2 Summary	58
23.1 You should now be able to	58
23.2 Exit checklist	58

III Phase 3 — AI Intrusion Detection System (IDS)	59
24 IDS Overview	60
24.1 Purpose	60
24.2 Architecture	60
24.3 Primary vs secondary models	60
24.4 Unprotected mode	60
24.5 Key files	61
25 IDS Dataset	62
25.1 Source	62
25.2 Label semantics	62
25.3 Class inventory (Tier A)	62
25.4 Files the trainers read	62
25.5 Balance notes	62
25.6 Loading for training	62
25.7 Quality checks	63
26 Feature Engineering	64
26.1 Physical features	64
26.2 Network features	64
26.3 What is dropped	64
26.4 Fusion	64
26.5 Why network helps	64
26.6 Live normalisation	64
26.7 Inspecting a feature row	65
27 Model Architecture	66
27.1 TinyMAV 1D-CNN (primary)	66
27.2 Why TinyMAV	66
27.3 LightGBM cascade (secondary)	66
27.4 Rules layer	66
28 Model Training	67
28.1 Prerequisites	67
28.2 Offline training	67
28.3 Dashboard / API training	67
28.4 Artefacts	67
28.5 Validation	67
28.6 Troubleshooting training	68
29 IDS Modes — Reactive, Proactive, Hybrid	69
29.1 Reactive	69
29.2 Proactive	69

29.3 Hybrid	69
29.4 Soft / prevent aliases	69
29.5 Engagement thresholds	70
30 Attack Detection Behaviour	71
30.1 Live scoring path	71
30.2 What “detection” looks like	71
30.3 Per-attack cues	72
30.4 False positives / negatives	72
30.5 How to validate detection without defence	72
31 Live Defence	73
31.1 Enablement	73
31.2 Proactive path	73
31.3 Reactive path	73
31.4 Hybrid path	73
31.5 Operator checklist	73
31.6 Success criteria	74
31.7 Abort coupling	74
32 End-to-End System Architecture	75
32.1 Three planes	75
32.2 Sysid map	75
32.3 Process map on DT host	75
32.4 Failure domains	75
32.5 Configuration single source	75
33 Performance Evaluation	76
33.1 Metrics tracked	76
33.2 How to gather	76
33.3 Suggested experimental matrix	77
33.4 Reporting tips	77
33.5 Latency	77
33.6 Where numbers live	77
34 Phase 3 Summary	78
34.1 You should now be able to	78
34.2 Exit checklist	78
35 Conclusion	79
35.1 What the laboratory demonstrates	79
35.2 Operational takeaway	79
36 Future Work	80
36.1 Near-term lab extensions	80

36.2 Research directions	80
36.3 Engineering debt worth addressing	80
Linux Command Reference	83
.1 Filesystem	83
.2 Packages	83
.3 Networking	83
.4 Processes	83
ROS 2 Command Reference (Optional Path)	84
.5 DDS notes	84
PX4, Gazebo, DDS, and MAVLink References	85
.6 PX4 SITL	85
.7 Gazebo Classic	85
.8 Remote lab scripts	85
.9 MAVLink / pymavlink on DT host	85
.10 Ports quick sheet	85
Folders, Environment, Topology, FAQ, Glossary	86
.11 Folder structure	86
.12 Environment variables	86
.13 Port assignments	86
.14 Network topology	87
.15 FAQ	87
.16 Glossary	87

List of Figures

1.1 Ubuntu Desktop after installation on UAV PC	2
3.1 Successful apt upgrade terminal	6
4.1 Terminal showing successful ping to 192.168.123.130	10
4.2 MAVLink data path with proactive gateway.	10
5.1 Terminal after successful ros-humble-desktop installation	13
5.2 ROS2 topic list showing /chatter	14
6.1 Gazebo Classic empty world after gzclient launch	15
7.1 Simplified PX4 SITL internal workflow.	17

7.2	PX4 SITL startup console alongside Gazebo Iris model	18
8.1	Command and telemetry communication workflow.	20
9.1	Gazebo simulation running with Iris model	21
9.2	PX4 SITL startup console	22
9.3	QGroundControl connection to SITL	22
9.4	Digital Twin dashboard during benign mission	23
10.1	Successful SSH BatchMode echo OK to UAV PC	25
12.1	High-level lab topology: Physical Twin, Digital Twin host, MAVLink path, and proactive gateway.	29
12.2	Dashboard first viewport with 3D twin and PT link LIVE	30
13.1	DT host System Settings / Network showing 192.168.123.x address	32
14.1	Browser opened at http://127.0.0.1:8000 after hard refresh	34
15.1	API /api/pt JSON with ok true	36
16.1	Digital Twin software architecture and data flow.	37
16.2	Browser network panel showing /api/state and /api/ids JSON	39
17.1	Finder/terminal listing of a gps_spoofing run_XX folder	41
18.1	Shared data-generation timeline for every scenario.	42
18.2	Dashboard pipeline progress across Tier A scenarios	43
20.1	Terminal running run_dashboard.sh with Uvicorn ready message	47
21.1	Digital Twin operational workflow.	48
21.2	Digital Twin with PT link LIVE and altitude HUD	49
21.3	Performance analytics panel with proactive block counts	52
22.1	Mission route with attack gate highlighted	54
22.2	GPS spoof — DT trail divergence	55
24.1	AI IDS pipeline from features to defence.	60
24.2	Dashboard showing IDS unprotected pill	61
25.1	Dataset explorer showing a labelled physical CSV	63
27.1	Primary CNN with optional LightGBM assist.	66
28.1	Training log showing cnn1d live train OK	68
29.1	Defence mode dropdown set to Proactive with Defence ON	70
30.1	Attack detection logs and IDS alert banner	71

31.1	PROACTIVE BLOCK log with latency	74
32.1	End-to-end actors in the cyber-range.	75
33.1	Performance analytics panel after a defended run	76
1	Appendix topology sketch.	87

List of Tables

2.1	Software inventory for the UAV–DT–IDS laboratory.	5
10.1	Phase 1 troubleshooting matrix.	24

About This Manual

This handbook documents the **UAV–Digital Twin–AI IDS** cyber-range in `uav-cyber-m1`. It targets beginners and experts who must reproduce the stack without oral guidance.

Laboratory topology

- **Physical Twin:** Ubuntu PC 192.168.123.130, SSH user `danish`, running PX4 SITL + Gazebo Classic.
- **Digital Twin host:** macOS/Linux running the `uav-cyber-m1` dashboard at `http://127.0.0.1:8000`, recorders, IDS, and gateway 127.0.0.1:19550. Lab clone path example: `/Users/danishvasan/Downloads`

Beginner Note: If you are new to networking, read Chapter 4 before installing anything.

How to read

Phase 1 builds the UAV workstation; Phase 2 the Digital Twin; Phase 3 the AI IDS and defence modes. Screenshot placeholders (`\figshot`) mark where to paste real lab captures. Architecture uses TikZ diagrams, not screenshots.

Safety

All attacks are for SITL only. Never inject against a real airframe.

Part I

Phase 1 — Building the UAV Environment

Chapter 1

System Requirements

1.1 Purpose

Confirm hardware/OS before installing robotics stacks. Weak machines produce Gazebo stutter that can be mistaken for cyber effects.

Beginner Note: You need **two** machines on one LAN: the UAV PC (physics) and the Digital Twin host (dashboard/IDS). A single laptop can host the DT while the UAV PC runs SITL.

1.2 Physical Twin (UAV PC)

Resource	Recommended	Notes
CPU	6+ cores	Gazebo + PX4
RAM	16–32 GB	GUI + SITL
GPU	Integrated OK	OpenGL for gzclient
Disk	64 GB+ SSD	PX4 build artefacts
Display	Local	PT GUI on UAV
Network	Gigabit to DT	192.168.123.0/24
OS	Ubuntu 20.04/22.04	[21]
Account	danish	passwordless SSH from DT

```
Ubuntu Desktop session on UAV PC (Wayland; GUI apps via XWayland)
$ echo 'Session type: wayland (gnome)'; loginctl show-session $(loginctl | awk '/danish/ {print $1; exit}') -p Type -p Desktop -p Display -p S
Session type: wayland (gnome)
Display=
Desktop=
Type=wayland
State=active

Open GUI windows (XWayland):
0x1200010 "Gazebo": ("gazebo" "gazebo") 1466x800+0+37 +70+64

Hostname: box
User: danish
DISPLAY for Gazebo: :0
```

Figure 1.1: Ubuntu Desktop after installation on UAV PC

1.3 Digital Twin / IDS host

4+ cores, 8–16 GB RAM (16+ with Torch), Python 3.11, same lab LAN, passwordless SSH to the UAV, browser for localhost:8000.

1.4 OS versions

UAV: Ubuntu 20.04/22.04 [21]. DT: macOS 13+ or Ubuntu 22.04.

1.5 Verification

```
ping -c 2 192.168.123.130
ssh danish@192.168.123.130 'nproc; free -h; echo OK'
```

Important: If SSH asks for a password every time, fix keys now — the dashboard cannot drive SITL interactively.

Chapter 2

Required Software

2.1 Master software inventory

Table 2.1 lists components used across Phase 1 (UAV workstation), Phase 2 (Digital Twin), and Phase 3 (IDS). Items marked *optional* support an extended ROS 2/DDS path; the operational lab path is MAVLink-centric via `pymavlink` [3, 2].

Beginner Note: Install Phase 1 tools on the UAV PC (Ubuntu). Install Python/FastAPI/-Torch on the Digital Twin host (typically macOS). You rarely need Gazebo on the Mac.

Software	Version (lab)	Purpose	Official site
Ubuntu	20.04/22.04	UAV OS	ubuntu.com [21]
PX4 Autopilot	main/stable	SITL autopilot	docs.px4.io [1]
Gazebo Classic	11.x	Physics + PT GUI	classic.gazebosim.org [4]
QGroundControl	latest	Manual GCS (close during capture)	qgroundcontrol.com [5]
Python	3.11	DT/IDS runtime	python.org
CMake / Ninja / GCC	distro	PX4 build toolchain	cmake.org [22]
Git	distro	Source control	git-scm.com
OpenSSH	distro	Passwordless UAV control	openssh.com
pymavlink	≥2.4.41	MAVLink TX/RX	GitHub py-mavlink [3]
FastAPI / Uvicorn	0.11x / 0.27+	Dashboard API	fastapi.tiangolo.com [11, 12]
three.js	CDN/ESM	3D Digital Twin	threejs.org [13]
PyTorch	2.x	TinyMAV 1D-CNN	pytorch.org [14]
LightGBM	≥4.0	Secondary IDS modalities	lightgbm.readthedocs.io [16]
scikit-learn	≥1.3	Metrics / encoders	scikit-learn.org [15]
pandas / NumPy	2.x / 1.24+	Dataset tables	pandas/numpy docs [17, 18]

Software	Version	Purpose	Official site
tcpdump / scapy	distro / ≥ 2.5	Network capture	tcpdump.org [20, 19]
paramiko	≥ 3.0	SSH automation	paramiko.org
ROS 2	Humble/Jazzy	<i>Optional</i> robotics bus	docs.ros.org [6]
Micro XRCE-DDS	current	<i>Optional</i> PX4 $\leftrightarrow$ ROS2	eProsima [7]
Fast DDS	current	<i>Optional</i> DDS implementation	eProsima [8]
MAVSDK	current	<i>Optional</i> high-level API	mavsdk.mavlink.io [9]
MAVROS	current	<i>Optional</i> ROS MAVLink bridge	mavros [10]

Table 2.1: Software inventory for the UAV-DT-IDS laboratory.

2.2 Where each package runs

Host	Must have	Nice to have
UAV PC	Ubuntu, PX4, Gazebo, SSH	QGC, ROS 2
DT host	Python 3.11, venv deps, browser	IDE / tectonics for docs

2.3 Python dependency pin file

The Digital Twin host installs:

```
cd /Users/danishvasan/Downloads/uav-cyber-ml
python3 -m venv .venv
.venv/bin/pip install -U pip
.venv/bin/pip install -r requirements.txt
```

Explanation: `requirements.txt` pins pymavlink, FastAPI, torch, lightgbm, scikit-learn, pandas, numpy, scapy, uvicorn, websockets, joblib, paramiko [25].

2.4 Verify imports

```
.venv/bin/python - <<'PY'
import pymavlink, fastapi, torch, lightgbm, pandas, sklearn, scapy
print("ok", torch.__version__)
PY
```

Tip: If Torch is heavy on a small laptop, still install it — TinyMAV is small, but the package provides the CNN runtime used by live IDS.

Common Mistake: Installing packages with system `pip` outside the `venv` causes `ModuleNotFoundError` when `run_dashboard.sh` uses `.venv`.

Chapter 3

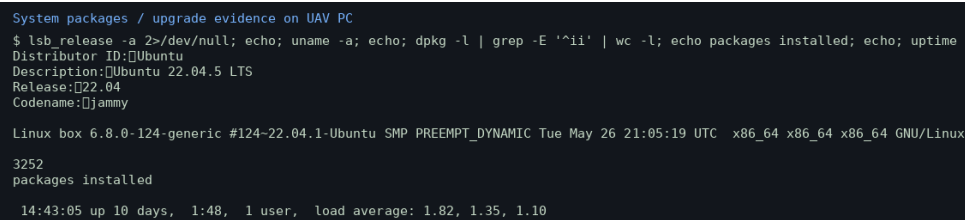
Linux Preparation

Prepare the Ubuntu UAV PC before PX4/Gazebo installs.

3.1 Update Ubuntu

```
sudo apt update && sudo apt upgrade -y
```

Explanation: Refresh indices and apply patches before adding robotics PPAs.



```
System packages / upgrade evidence on UAV PC
$ lsb_release -a 2>/dev/null; echo; uname -a; echo; dpkg -l | grep -E '^ii' | wc -l; echo packages installed; echo; uptime
Distributor ID: Ubuntu
Description: Ubuntu 22.04.5 LTS
Release: 22.04
Codename: jammy

Linux box 6.8.0-124-generic #124-22.04.1-Ubuntu SMP PREEMPT_DYNAMIC Tue May 26 21:05:19 UTC x86_64 x86_64 x86_64 GNU/Linux
3252
packages installed

14:43:05 up 10 days, 1:48, 1 user, load average: 1.82, 1.35, 1.10
```

Figure 3.1: Successful apt upgrade terminal

3.2 Build tools

```
sudo apt install -y build-essential cmake ninja-build git \
python3 python3-pip python3-venv python3-dev ccache pkg-config libssl-dev \
openssh-server
```

Explanation: Compilers and CMake are required to build PX4 [22].

3.3 Enable SSH server

```
sudo systemctl enable --now ssh
ip -br addr # note the 192.168.123.x address
```

Tip: Set a static IP 192.168.123.130 so config.py stays valid.

3.4 PATH

```
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc && source ~/.bashrc
```

3.5 Verification

```
gcc --version; cmake --version; git --version; python3 --version
```

3.6 Troubleshooting

apt lock errors — wait for unattended-upgrades. Permission denied — use sudo carefully. Disk full — `df -h` before multi-GB PX4 builds.

Chapter 4

Linux Networking for the Cyber-Range

This chapter explains networking from first principles. Later chapters assume you can ping the UAV PC, open SSH, and reason about UDP ports.

4.1 Why networking matters here

The Physical Twin (UAV PC) and Digital Twin host are **two computers**. All telemetry, attacks, SSH control, and IDS gateway traffic travel as IP packets. If the network is wrong, Gazebo may run on the UAV while the dashboard shows “Physical Twin unreachable” and the Digital Twin never moves.

Important: In this lab the UAV PC is typically 192.168.123.130 and the DT host must be on the same 192.168.123.0/24 network (Ethernet or lab Wi-Fi).

4.2 IP addresses

An **IPv4 address** identifies a host on a network, e.g. 192.168.123.130. The last number is the host; the first three (with a /24 mask) identify the subnet.

Beginner Note: Think of the subnet as a street name and the last octet as the house number. Hosts on different streets need a router to talk.

4.3 localhost

127.0.0.1 (localhost) means “this computer”. The dashboard binds 127.0.0.1:8000 and the proactive gateway binds 127.0.0.1:19550 so only local processes on the DT host can reach them.

4.4 Network interfaces

An **interface** is a NIC name such as eth0, en0, or wlan0.

```
# Ubuntu
ip -br addr
# macOS
ifconfig | grep -E '^[a-z]|inet '
```

Explanation: Lists interfaces and their IPv4 addresses so you can confirm you are on the lab subnet.

4.5 Ethernet vs Wi-Fi

Ethernet is preferred for SITL labs (lower jitter). Wi-Fi works if both hosts share the lab SSID/VLAN, but packet loss can freeze the twin.

4.6 Static vs dynamic IP

DHCP assigns addresses automatically. **Static IP** is fixed (recommended for the UAV PC so `config.py` stays correct).

4.7 Routing

```
# macOS: which interface reaches the UAV?
route get 192.168.123.130
# Ubuntu
ip route get 192.168.123.130
```

Explanation: Shows the outbound interface used toward the UAV; the lab recorder uses this to pick `NET_IFACE` for `tcpdump`.

4.8 Ports

A **port** selects an application on a host.

Port	Protocol	Use in this lab
22	TCP	SSH to UAV PC
8000	TCP	FastAPI dashboard (localhost)
14550	UDP	PX4 GCS telemetry broadcast
18570	UDP	GCS API / commands
19550	UDP	Proactive MAV gateway (localhost)

4.9 Firewalls

```
sudo ufw allow 22/tcp
sudo ufw status
```

Explanation: If SSH times out while ping works, a firewall may be blocking TCP 22.

4.10 ping

```
ping -c 3 192.168.123.130
```

Explanation: ICMP reachability test. 0% loss and ~1 ms RTT is healthy on a lab LAN.

```
Terminal showing successful ping to 192.168.123.130
PING 192.168.123.130 (192.168.123.130): 56 data bytes
64 bytes from 192.168.123.130: icmp_seq=0 ttl=64 time=1.105 ms
64 bytes from 192.168.123.130: icmp_seq=1 ttl=64 time=1.124 ms
64 bytes from 192.168.123.130: icmp_seq=2 ttl=64 time=0.969 ms
64 bytes from 192.168.123.130: icmp_seq=3 ttl=64 time=0.974 ms

--- 192.168.123.130 ping statistics ---
4 packets transmitted, 4 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 0.969/1.043/1.124/0.072 ms
```

Figure 4.1: Terminal showing successful ping to 192.168.123.130

4.11 SSH and SCP

```
ssh danish@192.168.123.130
scp file.txt danish@192.168.123.130:~/
```

Explanation: SSH opens a remote shell; SCP copies files. The lab requires passwordless SSH keys for non-interactive SITL control.

Best Practice: Generate a key once with `ssh-keygen -t ed25519` and install it with `ssh-copy-id danish@192.168.123.130`.

4.12 ROS 2 / DDS networking (optional path)

ROS 2 uses DDS and a **Domain ID**. Hosts must share `ROS_DOMAIN_ID` and be multicast-reachable [6, 8]. This lab's operational path does **not** require ROS 2; MAVLink UDP is enough.

4.13 MAVLink networking

MAVLink messages are usually carried in UDP datagrams [2]. The DT host sends commands via the local gateway (19550), which forwards allowed frames to the UAV (14550). Telemetry returns on 14550 to the physical recorder.

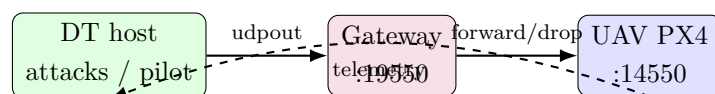


Figure 4.2: MAVLink data path with proactive gateway.

4.14 Troubleshooting and verification

Symptom	Cause	Fix
100% ping loss	Wrong Wi-Fi / VLAN	Join lab 192.168.123.x
SSH timeout	Firewall / no keys	Open TCP 22; ssh-copy-id
Dashboard PT offline	UAV IP changed	Update UAV_HOST
No twin motion	14550 busy (QGC)	Close QGroundControl

1. `ping -c 2 192.168.123.130` succeeds.
2. `ssh danish@192.168.123.130 echo ok` prints ok.
3. `curl -s http://127.0.0.1:8000/api/pt` shows ok: true after Start sim.

Chapter 5

Installing ROS 2

5.1 Before installation — what is ROS 2?

ROS 2 (Robot Operating System 2) is middleware for robotics software [6]. Programs called **nodes** publish/subscribe **topics**, call **services**, and share a **DDS** databus.

5.1.1 Role in a UAV lab

- Optional bridge between PX4 and high-level autonomy (offboard setpoints).
- Visualisation (**rviz2**), bag recording, launch files.

Important: The operational `uav-cyber-ml` Digital Twin does **not** require ROS 2. Telemetry and attacks use MAVLink via `pymavlink` [3]. Install ROS 2 if you extend the lab toward ROS offboard or XRCE-DDS.

5.1.2 How ROS 2 relates to other components

Component	Relationship
PX4	May expose uORB via Micro XRCE-DDS agent [7]
Gazebo	Can publish simulated sensors into ROS topics
MAVROS	Translates MAVLink ↔ ROS topics [10]
Digital Twin (this lab)	Uses MAVLink/HTTP instead of ROS topics

5.1.3 Supported versions

Ubuntu 22.04 → ROS 2 Humble; Ubuntu 24.04 → Jazzy. Mixing distros causes obscure library errors.

5.2 Installation (Humble on Ubuntu 22.04)

```
sudo apt update
sudo apt install -y software-properties-common curl gnupg lsb-release
sudo add-apt-repository universe
```

```

sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
  -o /usr/share/keyrings/ros-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] \
http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo $UBUNTU_CODENAME) main" \
| sudo tee /etc/apt/sources.list.d/ros2.list >/dev/null
sudo apt update
sudo apt install -y ros-humble-desktop python3-colcon-common-extensions

```

Explanation: Adds the ROS apt repository, installs the desktop metapackage (tools + RViz), and colcon for building workspaces.

```

ROS 2 Humble packages on UAV PC
$ dpkg -l | grep -E 'ros-humble-desktop|ros-humble-ros-base' | head -10; echo; which ros2; ros2 --help 2>&1 | head -3; echo; ls /opt/ros/humble
ii  ros-humble-desktop      0.10.0-1jammy.20260612.213429      amd64      A package which extends 'r
ii  ros-humble-ros-base     0.10.0-1jammy.20260607.081808      amd64      A package which extends 'r

/bin/sh: 1: ros2: not found

bin
cmake
include
lib
local
local_setup.bash
local_setup.sh
local_setup_util.py
local_setup.zsh
opt

```

Figure 5.1: Terminal after successful ros-humble-desktop installation

Common Mistake: Forgetting source /opt/ros/humble/setup.bash makes every ros2 command fail with “command not found”.

5.3 Environment setup

```

echo 'source /opt/ros/humble/setup.bash' >> ~/.bashrc
source /opt/ros/humble/setup.bash
echo $ROS_DISTRO

```

Explanation: Expected output includes humble.

5.4 Verification and testing

```

ros2 doctor
ros2 run demo_nodes_cpp talker
# other terminal:
ros2 run demo_nodes_cpp listener
ros2 topic list

```

Explanation: Talker/listener proves DDS discovery works on the machine.

```

ROS 2 topic check on UAV PC
$ bash -lc 'source /opt/ros/humble/setup.bash 2>/dev/null; ros2 topic list 2>&1 | head -40'
/fmu/in/actuator_motors
/fmu/in/actuator_servos
/fmu/in/arming_check_reply_v1
/fmu/in/aux_global_position
/fmu/in/config_control_setpoints
/fmu/in/config_overrides_request
/fmu/in/distance_sensor
/fmu/in/fixed_wing_lateral_setpoint
/fmu/in/fixed_wing_longitudinal_setpoint
/fmu/in/goto_setpoint
/fmu/in/lateral_control_configuration
/fmu/in/longitudinal_control_configuration
/fmu/in/manual_control_input
/fmu/in/message_format_request
/fmu/in/mode_completed
/fmu/in/obstacle_distance
/fmu/in/offboard_control_mode
/fmu/in/onboard_computer_status
/fmu/in/register_ext_component_request
/fmu/in/rover_attitude_setpoint
/fmu/in/rover_position_setpoint
/fmu/in/rover_rate_setpoint
/fmu/in/rover_steering_setpoint
/fmu/in/rover_throttle_setpoint
/fmu/in/rover_velocity_setpoint
/fmu/in/sensor_optical_flow
/fmu/in/telemetry_status
/fmu/in/trajectory_setpoint
/fmu/in/unregister_ext_component
/fmu/in/vehicle_attitude_setpoint_v1
/fmu/in/vehicle_command
/fmu/in/vehicle_command_mode_executor
/fmu/in/vehicle_mocap_odometry
/fmu/in/vehicle_rates_setpoint
/fmu/in/vehicle_thrust_setpoint
/fmu/in/vehicle_torque_setpoint
/fmu/in/vehicle_visual_odometry
/fmu/out/airspeed_validated_v1
/fmu/out/arming_check_request
/fmu/out/battery_status_v1

```

Figure 5.2: ROS2 topic list showing /chatter

5.5 Uninstall (optional)

```

sudo apt remove --purge '~nros-humble-.*' -y || true
sudo apt autoremove -y

```

5.6 Troubleshooting

Error	Cause	Fix
command not found	setup not sourced	source setup.bash
No topics across PCs	Domain/firewall	Match ROS_DOMAIN_ID; allow multicast
apt key errors	Old keyring	Re-download ros.key

Chapter 6

Installing Gazebo

6.1 Before installation

Gazebo Classic separates `gzserver` (physics) from `gzclient` (GUI) [4]. It is the visual Physical Twin when paired with PX4 SITL.

6.1.1 Dependencies and compatibility

Use the Gazebo Classic version pulled by PX4's Ubuntu setup script to avoid plugin ABI mismatches [1].

6.2 Installation via PX4 dependency script

```
cd ~
git clone https://github.com/PX4/PX4-Autopilot.git --recursive
bash ./PX4-Autopilot/Tools/setup/ubuntu.sh
```

Explanation: Installs compilers, Gazebo Classic, and PX4 build dependencies.

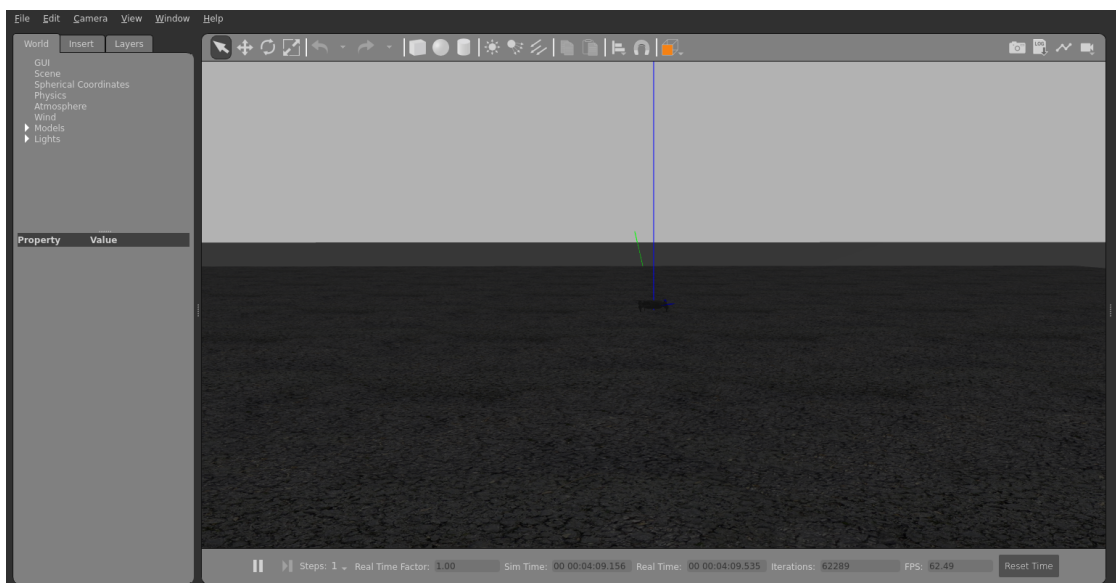


Figure 6.1: Gazebo Classic empty world after `gzclient` launch

6.3 Verification

```
which gzserver gzclient  
gzserver --verbose &  
sleep 2; gzclient &
```

Explanation: Server runs physics; client shows the 3D window on DISPLAY.

Warning: On the UAV PC, Gazebo GUI needs a local display (DISPLAY=:0). Headless SSH alone will not show the Physical Twin window.

6.4 Common failures

- Black window — GPU/GL drivers.
- Immediate crash — incomplete PX4 deps; re-run ubuntu.sh.
- Multiple gzserver — kill leftovers: `pkill gzserver; pkill gzclient`.

6.5 Uninstall notes

Prefer leaving PX4-managed Gazebo installed; purging ROS/Gazebo packages can break SITL. Only remove if you intentionally rebuild the workstation.

Chapter 7

Installing PX4

7.1 Before installation — PX4 concepts

PX4 is an open-source flight stack: estimators (EKF2), controllers, mixers, and a MAVLink interface [1]. **SITL** runs that stack on Linux with simulated sensors driving Gazebo motors/joints.

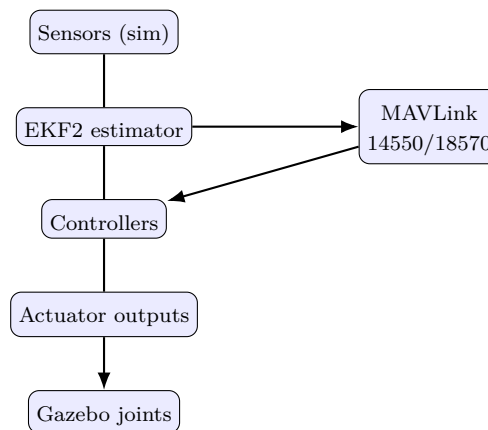


Figure 7.1: Simplified PX4 SITL internal workflow.

7.2 Clone and build

```
cd ~
git clone https://github.com/PX4/PX4-Autopilot.git --recursive
cd PX4-Autopilot
make px4_sitl gazebo-classic
```

Explanation: First build downloads toolchains and may take 30–90 minutes. Success ends with Gazebo + PX4 console.

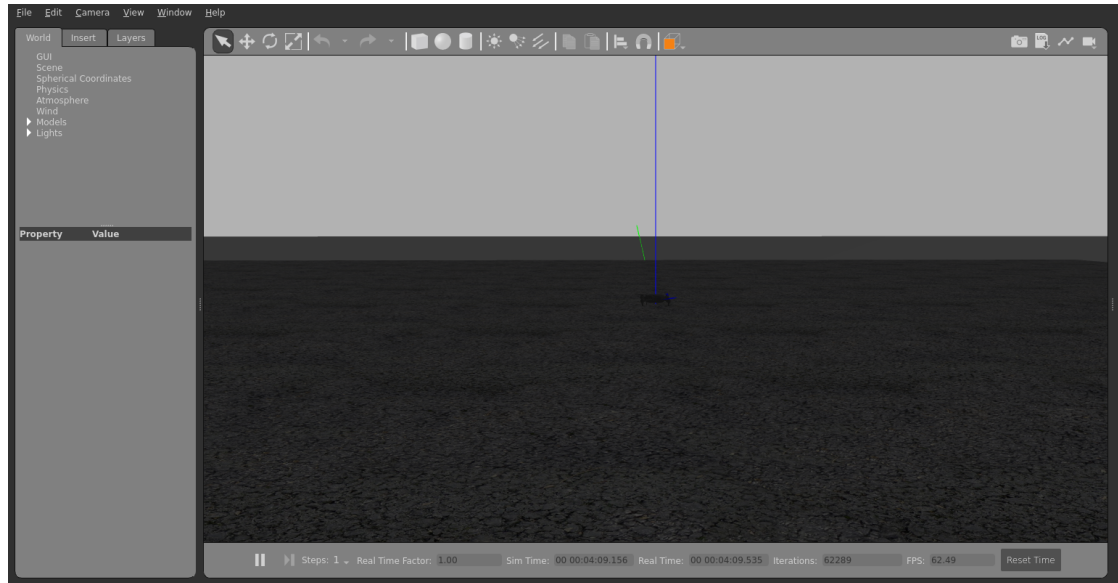


Figure 7.2: PX4 SITL startup console alongside Gazebo Iris model

7.3 Lab remote start

```
ssh danish@192.168.123.130 'cd ~/uav_cyber_testbed && ./scripts/start_sitl_baseline.sh'
```

Explanation: Used by `ssh_control.start_sitl()` from the Digital Twin host.

7.4 Verification

```
ssh danish@192.168.123.130 'pgrep -a px4 | head; pgrep -x gzserver; pgrep -x gzclient'
```

7.5 Troubleshooting

Symptom	Cause	Fix
Build fails on cmake	Missing deps	Re-run Tools/setup/ubuntu.sh
No drone in Gazebo	Wrong make target	Use gazebo-classic target
Arm denied	Prearm checks	Wait EKF; check PX4 console

Chapter 8

Communication Protocols in Depth

8.1 MAVLink from first principles

MAVLink is a lightweight message protocol for drones [2]. Each message has a msgid (e.g. 0 HEARTBEAT, 76 COMMAND_LONG, 232 GPS_INPUT) and a **system id** identifying the sender.

Beginner Note: System id 1 is usually the autopilot. In this lab the Mac spoofs additional GCS identities: controller 252, attacker 250, defender 249.

8.2 Framing

MAVLink v1 starts with 0xFE; v2 with 0xFD. The gateway parser in `ids/mav_gateway.py` extracts sysid and msgid from both.

8.3 Lab ports

Port	Role	Notes
14550	GCS broadcast telemetry	Physical recorder / twin bridge
18570	GCS API / command	Streams + some command paths
19550	Mac-local proactive gateway	Filters attacker traffic
8000	Dashboard HTTP/WS	localhost only
14540	RX telemetry	config RX_PORT
14580	Offboard API	config OFFBOARD_PORT

8.4 Trusted vs attacker traffic

Trusted sysids always pass the gateway. Attacker sysid 250 is subject to dangerous-msgid drops when Defence is ON in proactive/hybrid modes.

8.5 pymavlink

Python bindings used by pilot, attacks, and recorders [3]. Links typically `udpout:127.0.0.1:19550` when the gateway is enabled.

8.6 DDS / Micro XRCE-DDS / Fast DDS

Optional ROS 2 path [7, 8]. Not required for the operational IDS.

8.7 MAVSDK and MAVROS

Higher-level alternatives [9, 10]; this lab standardises on pymavlink.

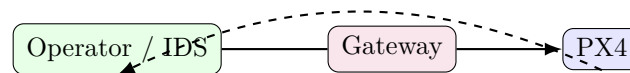


Figure 8.1: Command and telemetry communication workflow.

8.8 Troubleshooting

Wrong sysid trust lists cause either dropped benign commands or undropped attacks. Verify `config.py` identities match gateway trusted set.

Common Mistake: Setting `MAV_GATEWAY=0` bypasses prevention — useful for ablations, disastrous if you assume proactive defence is active.

Chapter 9

Running the First UAV Simulation

9.1 Goals

Boot PX4+Gazebo, confirm the Physical Twin GUI, then drive a benign mission from the Digital Twin dashboard.

9.2 On the UAV PC (local first flight)

```
cd ~/PX4-Autopilot
make px4_sitl gazebo-classic
```

Explanation: Starts SITL. Wait until the Iris model appears and the PX4 shell accepts commands.

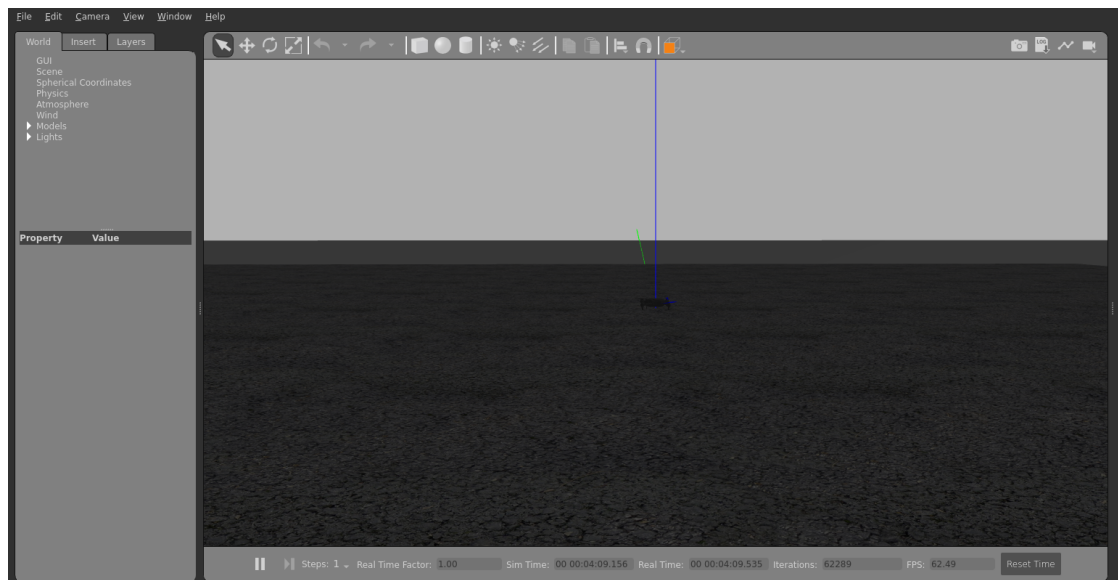


Figure 9.1: Gazebo simulation running with Iris model

```

UAV PC SITL status -- real capture Wed Jul 29 02:41:50 PM +03 2026
host: box
  PID %CPU %MEM    ELAPSED CMD
  532960 12.3  0.0      01:45:12 /home/danish/PX4-Autopilot/build/px4_sitl_default/bin/px4 /home/danish/PX4-Autopilot/build/px4_sitl_default/etc
  ...
532732 gzserver /home/danish/PX4-Autopilot/Tools/simulation/gazebo-classic/sitl_gazebo-classic/worlds/empty.world
532909 gzclient
532959 gzclient --verbose --gui-client-plugin libgazebo_user_camera_plugin.so
556656 /bin/sh -c pgrep -af 'gzserver|gzclient' | head -8
  ...
recent log dir hint:
-rw-rw-r-- 1 danish danish 0 Jul 29 12:56 /tmp/px4_lock-0
srwxrwxr-x 1 danish danish 0 Jul 29 12:56 /tmp/px4-sock-0

```

Figure 9.2: PX4 SITL startup console

9.3 Optional QGroundControl

Connect QGC to UDP 14550 [5], verify telemetry, then **close QGC** before dataset capture.

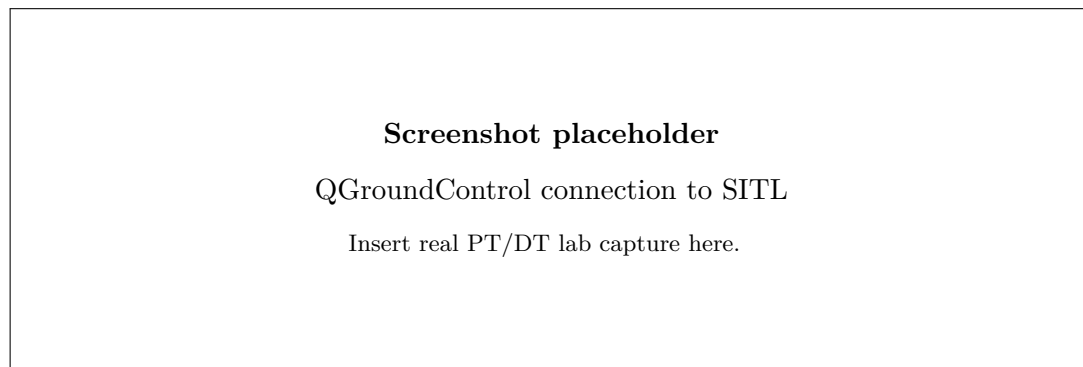


Figure 9.3: QGroundControl connection to SITL

9.4 From the Digital Twin host

```

cd uav-cyber-ml
./run_dashboard.sh
# open http://127.0.0.1:8000
# Click Start sim, wait until ready, Run benign

```

Explanation: Dashboard SSHs to the UAV, ensures SITL, flies the shared mission, streams the DT.

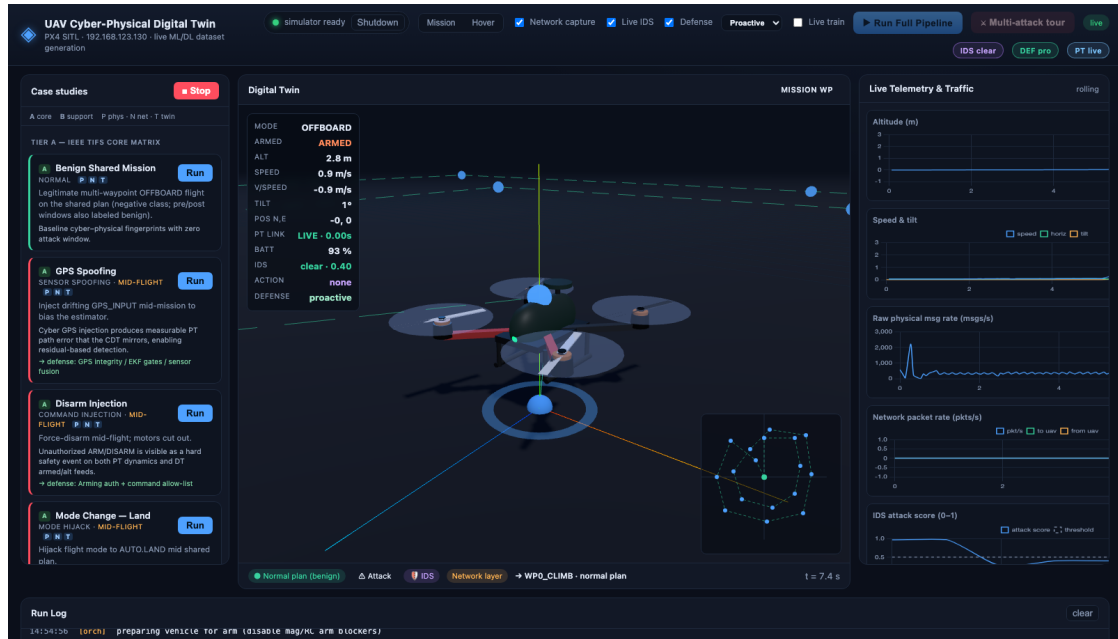


Figure 9.4: Digital Twin dashboard during benign mission

9.5 Landing and shutdown

Use dashboard Stop, or land via orchestrator wind-down. To hard-stop SITL:

```
ssh danish@192.168.123.130 'pkill -f px4; pkill -f gzserver; pkill -f gzclient'
```

9.6 Troubleshooting

- Start sim fails — check Chapter 4 reachability.
- Twin grounded — port 14550 conflict with QGC.
- Scenario skips — SITL not ready; read orch logs.

Chapter 10

Phase 1 Troubleshooting

This chapter is the Phase 1 first-aid kit. Work top-to-bottom: network, then SSH, then display/Gazebo, then PX4 build, then ports.

Beginner Note: Most “sim is broken” tickets are actually “Mac cannot reach 192.168.123.130” or “QGroundControl is holding UDP 14550”.

10.1 Symptom matrix

Symptom	Likely cause	Fix
SSH timeout	Mac not on lab LAN	Join 192.168.123.0/24; check cable/Wi-Fi
Ping works, SSH fails	Firewall / sshd down	<code>sudo ufw allow 22/tcp</code> ; restart ssh
Permission denied (publickey)	Missing SSH key	<code>ssh-copy-id danish@192.168.123.130</code>
<code>gzclient</code> missing / black	<code>DISPLAY</code> unset / no X	Run on UAV console; <code>export DISPLAY=:0</code>
PX4 build fails	Missing deps	Re-run <code>ubuntu.sh</code> [1]
Port 14550 busy	QGC open	Close QGroundControl during capture [5]
ROS 2 not found	Setup not sourced	<code>source /opt/ros/\$ROS_DISTRO/setup.bash</code>
Gazebo freezes	GPU / OpenGL	Prefer onboard GPU; reduce GUI plugins
SITL exits immediately	Bad PX4 path / world	Confirm 7PX4-Autopilot and model
Dashboard PT unreachable	Wrong UAV_HOST	Match <code>config.py</code> / env to real IP

Table 10.1: Phase 1 troubleshooting matrix.

10.2 Network and SSH checklist

```
# On Digital Twin host (Mac)
ping -c 3 192.168.123.130
route get 192.168.123.130 # macOS
ssh -o BatchMode=yes danish@192.168.123.130 'hostname; echo OK'
```

Explanation: BatchMode fails fast if keys are missing — fix keys before debugging Gazebo.

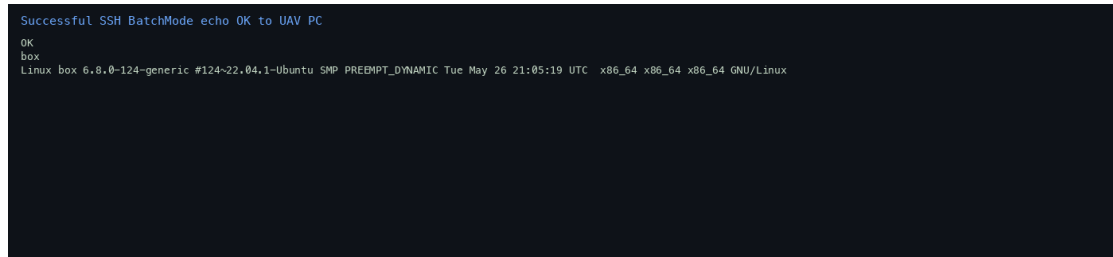


Figure 10.1: Successful SSH BatchMode echo OK to UAV PC

10.3 Display and Gazebo

```
# On UAV PC console session
echo $DISPLAY
export DISPLAY=:0
gzserver --verbose &
gzclient
```

Warning: Headless SSH sessions often lack a usable DISPLAY. Launch Gazebo from the UAV desktop session, or configure a virtual framebuffer only if you know the lab policy.

10.4 PX4 build failures

```
cd ~/PX4-Autopilot
bash ./Tools/setup/ubuntu.sh
make px4_sitl gazebo-classic
```

Tip: If the first build died mid-download, re-run `make`; CMake usually resumes. Disk full is a common silent killer — check with `df -h`.

10.5 Port conflicts

```
# On Mac --- who owns 14550?
sudo lsof -nP -iUDP:14550 || true
# On UAV
ss -ulnp | grep -E '14550|18570' || netstat -ulnp | grep 14550
```

Common Mistake: Leaving QGroundControl connected during dataset capture steals the GCS telemetry peer and breaks the physical recorder bind on 14550.

10.6 ROS 2 optional path

Only needed if you follow the educational DDS chapters. Operational DT/IDS does not require ROS 2 [6].

Chapter 11

Phase 1 Summary

Phase 1 stands up the Physical Twin: Ubuntu UAV PC, networking, optional ROS 2, Gazebo Classic, PX4 SITL, and a first successful flight.

11.1 You should now be able to

1. Ping and SSH to `danish@192.168.123.130`.
2. Launch PX4 SITL with Gazebo and see the Iris model.
3. Explain why QGroundControl must release UDP 14550 during capture.
4. Distinguish the optional ROS 2/DDS path from the operational MAVLink path.

11.2 Exit checklist

- `ssh danish@192.168.123.130 echo ok` succeeds without a password prompt.
- `make px4_sitl gazebo-classic` (or lab script) brings up SITL.
- Chapter 10 symptoms are understood before Phase 2.

Best Practice: Do not proceed to the Digital Twin until passwordless SSH works. Every dashboard “Start sim” click depends on it.

Part II

Phase 2 — Digital Twin Development

Chapter 12

Digital Twin Overview

A **Digital Twin (DT)** is a synchronised cyber representation of a physical asset [23]. In this lab the DT is a browser-based 3D vehicle driven by live PX4 `LOCAL_POSITION_NED` / `ATTITUDE` telemetry, plus live graphs and IDS overlays [13].

Unlike pure simulation, the DT **does not replace** Gazebo: Gazebo+PX4 remain the Physical Twin; the DT mirrors state for monitoring, labelling, and defence research [24].

Beginner Note: Think “instrument panel + 3D mirror”, not “second simulator”. If Gazebo stops, the twin freezes — it has no independent physics.

12.1 Lab topology

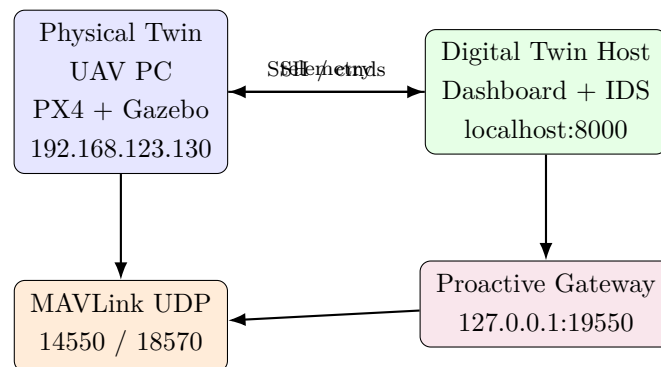


Figure 12.1: High-level lab topology: Physical Twin, Digital Twin host, MAVLink path, and proactive gateway.

12.2 What the DT shows

- 3D quadcopter pose, trail, and top-down mini-map.
- Armed/mode/altitude/speed HUD.
- Live physical and network charts with attack-window shading.
- IDS score pills, defence mode, proactive block counters.

- Dataset explorer for historical raw/processed CSVs.

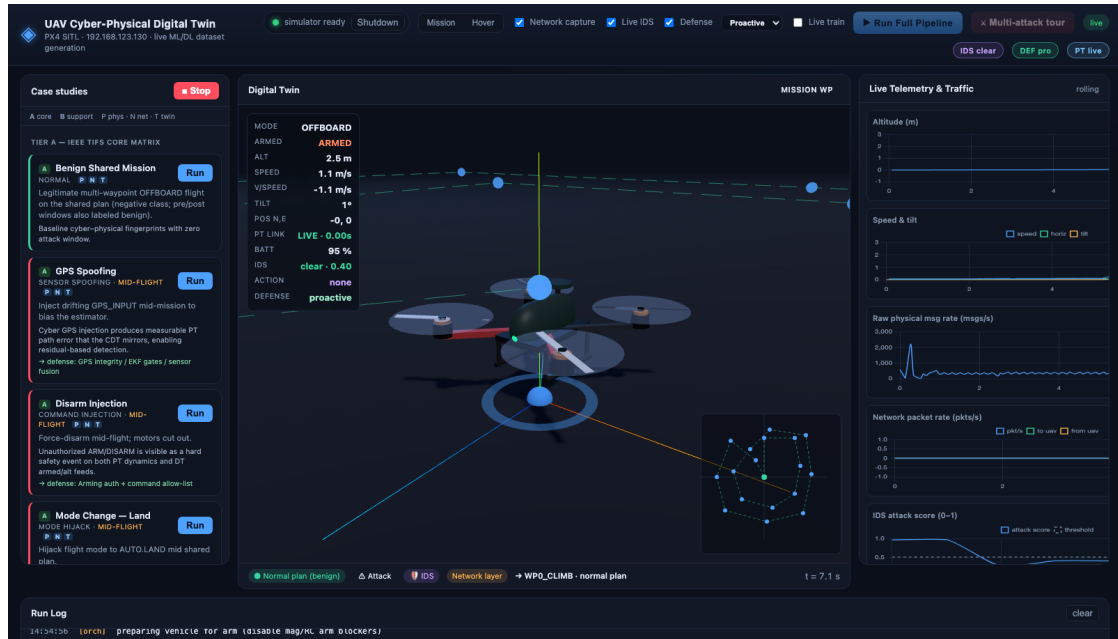


Figure 12.2: Dashboard first viewport with 3D twin and PT link LIVE

12.3 Advantages

- Aligned physical, network, and twin observations on one shared mission.
- One-click scenarios for reproducible cyber-range runs.
- Defence research without rewriting PX4.

12.4 Limitations

DT fidelity is bounded by telemetry rate, gateway filtering, and UI refresh; it is not a second physics engine. Proactive drops can make attacks *invisible* on the vehicle — which is correct for prevention studies but wrong for signature collection (use Defence OFF).

12.5 Relationship to ROS 2

An educational ROS 2/DDS twin is optional. The operational `uav-cyber-m1` twin is MAVLink + FastAPI + Three.js only.

Chapter 13

Digital Twin Host Hardware

The Digital Twin host is the machine that runs the FastAPI dashboard, Three.js viewer, recorders, IDS, and proactive MAVLink gateway. In this laboratory it is typically a macOS workstation on the same LAN as the UAV PC.

13.1 Role of the DT host

- SSH-control PX4+Gazebo on 192.168.123.130 (user `danish`).
- Bind UDP 14550 for physical telemetry (close QGC during capture).
- Run `tcpdump` on the interface that routes to the UAV (`NET_IFACE`).
- Serve `http://127.0.0.1:8000` and gateway `127.0.0.1:19550`.

Beginner Note: The DT host does **not** need Gazebo or PX4 installed locally. Physics stays on the UAV PC; the twin only mirrors MAVLink state.

13.2 Recommended hardware

Resource	Guidance
CPU	Modern multi-core (Apple Silicon or x86_64)
RAM	≥ 16 GB (32 GB comfortable with Torch + browser)
Disk	≥ 40 GB free for datasets + <code>.venv</code> + Torch
Network	Gigabit Ethernet preferred to lab switch/UAV
Display	Any; browser must reach localhost:8000

13.3 Network attachment

```
# Confirm DT host is on 192.168.123.0/24
ifconfig | grep -A2 'inet 192.168.123'
ping -c 2 192.168.123.130
```

Important: If the DT host is on a different subnet, SSH and MAVLink will fail even when Gazebo looks healthy on the UAV monitor.

13.4 Privileges

Network capture needs elevated rights for tcpdump [20]. Prefer `./scripts/enable_network_capture.sh` or the dashboard setup-capture endpoint; otherwise uncheck Network capture for physical-only runs.

```
DT host network interface 192.168.123.x
[inet 192.168.123.123 netmask 0xffffffff broadcast 192.168.123.255
[nd6 options=201<PERFORMNUD,DAD>
[media: autoselect (100baseTX <full-duplex,flow-controls>)
[status: active
[bridge0: flags=8063<UP,BROADCAST,SMART,RUNNING,SIMPLEX,MULTICAST> mtu 1500
[options=63<RXCSUM,TXCSUM,TSO4,TSO6>

  route to: 192.168.123.130
destination: 192.168.123.130
interface: en0
  flags: <UP,HOST,DOWN,LLINFO,WASCLONED,IFSCOPE,IFREF>
recvpipe sendpipe ssthresh rtt,msec rttvar hopcount      mtu      expire
      0         0 22243214         6         8         0    1500    1199
```

Figure 13.1: DT host System Settings / Network showing 192.168.123.x address

Chapter 14

Software Installation on the Digital Twin Host

14.1 Before installation

The DT host runs Python, FastAPI, the 3D twin, recorders, IDS, and SSH control. It does **not** need Gazebo locally.

Beginner Note: Project root in this lab: `/Users/danishvasan/Downloads/uav-cyber-ml`. Adjust if your clone path differs.

14.2 Install

```
cd /Users/danishvasan/Downloads/uav-cyber-ml
python3 -m venv .venv
.venv/bin/pip install -U pip
.venv/bin/pip install -r requirements.txt
chmod +x run_dashboard.sh
ssh-copy-id danish@192.168.123.130
```

Explanation: Creates an isolated venv and installs pymavlink, FastAPI, torch, lightgbm, pandas, etc. [11, 14, 16].

14.3 Network capture privilege

```
./scripts/enable_network_capture.sh
# or uncheck Network capture in the UI
```

Explanation: tcpdump needs elevated privileges on macOS/Linux [20].

14.4 Launch verification

```
./run_dashboard.sh
# hard-refresh browser
open http://127.0.0.1:8000 # macOS
curl -s http://127.0.0.1:8000/api/ids | python3 -m json.tool | head
```

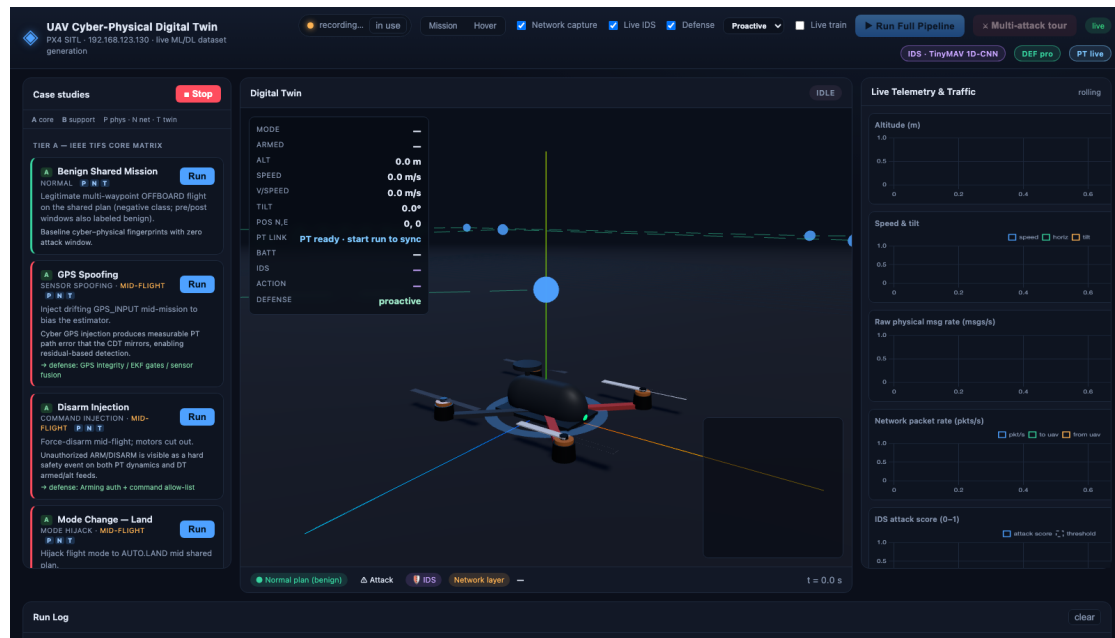


Figure 14.1: Browser opened at <http://127.0.0.1:8000> after hard refresh

14.5 Troubleshooting

- `ModuleNotFoundError` — use `.venv/bin/python`.
- SSH fails — Chapter 4.
- Port 8000 busy — stop the other Uvicorn; or change bind in the launch script.
- Torch install fails — install CPU wheel from pytorch.org then re-run requirements.

Best Practice: After `git pull`, hard-refresh the browser so `app.js` updates.

Chapter 15

Connecting Physical Twin and Digital Twin

15.1 Goal

Prove the DT host can start SITL on the UAV, receive telemetry on UDP 14550, and render a live twin at localhost:8000.

15.2 Preconditions

- Passwordless SSH: `ssh danish@192.168.123.130 echo ok`
- Dashboard running: `./run_dashboard.sh`
- QGroundControl closed (frees 14550)

15.3 Connect workflow

1. Click **Start sim** (or call SSH start from CLI helpers).
2. Wait until Gazebo Iris appears on the UAV display.
3. Confirm PT link LIVE and twin pose updates.
4. Optionally run **benign** once as a smoke test.

15.4 API checks

```
curl -s http://127.0.0.1:8000/api/pt | python3 -m json.tool
curl -s http://127.0.0.1:8000/api/state | python3 -m json.tool | head -40
```

Explanation: PT JSON should eventually show px4/gzserver/gzclient counts > 0 after Start sim.

```
GET /api/pt JSON with ok true (live lab)
{
  "px4": 7,
  "gzserver": 1,
  "gzclient": 2,
  "display": "0",
  "raw": "PX4=7\nGZSERVER=1\nGZCLIENT=2\nDISPLAY=:0\nPOSE=-9.1e-05 2.9e-05 0.104576 0.000108 -0.001693 6e-06\n",
  "pose": {
    "x": -9.1e-05,
    "y": 2.9e-05,
    "z": 0.104576
  },
  "ok": true,
  "gui": true,
  "flying": false,
  "where": "UAV display :0 @ 192.168.123.130 (Gazebo GUI is on the UAV PC, not the Mac browser)"
}
```

Figure 15.1: API /api/pt JSON with ok true

15.5 Data path reminder

Telemetry: UAV:14550 → physical recorder → WebSocket → Three.js.

Commands: UI/orchestrator → gateway:19550 → UAV.

15.6 Troubleshooting

- PT unreachable — ping/SSH; verify UAV_HOST.
- Twin frozen at origin — no 14550 traffic; close QGC; restart sim.
- Start sim hangs — SITL script path on UAV; DISPLAY for gzclient.

Warning: Never point attackers at a real vehicle. This connection path is SITL-lab only.

Chapter 16

Digital Twin Architecture

This chapter maps modules in `uav-cyber-ml` to runtime data paths.

16.1 Block architecture

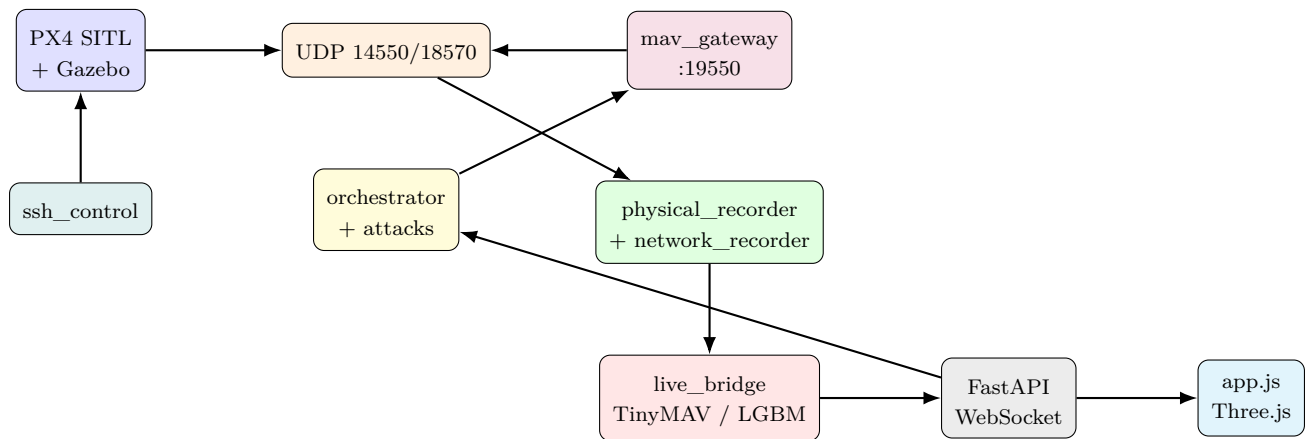


Figure 16.1: Digital Twin software architecture and data flow.

16.2 Module map

- `dashboard/server.py` — FastAPI routes, WebSocket bus, run control.
- `dashboard/static/app.js` — Three.js twin + charts + UI actions.
- `recorders/physical_recorder.py` — binds UDP 14550; emits phys samples.
- `recorders/network_recorder.py` / `live_network.py` — tcpdump windows.
- `recorders/twin_bridge.py` — landing handoff keepalive.
- `ids/mav_gateway.py` — proactive drop policy before PX4.
- `ids/live_bridge.py` — online scoring + metrics + defence hooks.
- `orchestrator.py` — scenario lifecycle and labelling.

- `attacks/suite.py`, `attacks/benign.py` — injectors and pilot.
- `config.py` — hosts, ports, mission plan, defence defaults.

16.3 Control vs telemetry planes

Telemetry plane: PX4 → UDP 14550 → physical recorder → UI/IDS.

Control plane: UI/orchestrator → gateway :19550 → UAV :14550/:18570.

Management plane: SSH from DT host → start/stop SITL processes.

Important: Trusted sysids (controller 252, defender 249, recorder 254) pass the gateway; attacker sysid 250 is subject to dangerous-msgid filtering when Defence is ON in proactive/hybrid modes.

16.4 API surface (selected)

Route	Method	Role
<code>/api/pt</code>	GET	Physical Twin process status
<code>/api/state</code>	GET	Run/busy/mission snapshot
<code>/api/ids</code>	GET/POST	IDS status and defence controls
<code>/api/train</code>	GET/POST	Offline/live train triggers
<code>/api/run</code>	POST	Start a scenario
<code>/api/scenarios</code>	GET	Scenario catalogue

```
Browser Network-equivalent JSON: /api/state and /api/ids
{
  "state_status": "idle",
  "sim": "ready",
  "pt_ok": true,
  "ids_ready": true,
  "defense_mode": "proactive",
  "n_alerts": 3,
  "metrics": {
    "tp": 1,
    "fp": 2,
    "tn": 44,
    "fn": 0,
    "precision": 0.3333,
    "recall": 1.0,
    "f1": 0.5,
    "accuracy": 0.9574,
    "fpr": 0.0435,
    "n_scores": 47,
    "mean_detection_delay_s": 0.41,
    "last_detection_delay_s": 0.41,
    "mean_mitigation_delay_s": 1.294,
    "last_mitigation_delay_s": 1.294,
    "detections": 1,
    "alerts_total": 3,
    "defenses": 1,
    "defense_success": 1,
    "defense_success_rate": 1.0,
    "mission_resume_ok": 1,
    "mission_resume_fail": 0,
    "mission_resume_rate": 1.0,
    "gt_attack": false,
    "gt_class": null,
    "mean_score": 0.435,
    "alerts_by_class": {
      "command_flood_dos": 2,
      "param_injection": 1
    },
    "preventions_by_class": {
      "param_injection": 1
    },
    "prevention_ok_by_class": {
      "param_injection": 1
    },
    "gt_windows_by_class": {
      "gps_spoofing": 1
    },
    "detected_windows_by_class": {
      "gps_spoofing": 1
    },
    "proactive_blocks": 0,
    "reactive_recovers": 0,
    "proactive_ok": 1,
    "reactive_ok": 0,
    "proactive_by_class": {},
    "reactive_by_class": {},
    "mean_proactive_block_ms": null,
  }
}
```

Figure 16.2: Browser network panel showing /api/state and /api/ids JSON

Best Practice: When debugging architecture issues, curl the JSON APIs before assuming the Three.js front end is wrong.

Chapter 17

Data Collection

Data collection turns a live flight into labelled CSVs under `datasets/runs/<scenario>/run_XX/`.

Beginner Note: Collection is automatic when you run a scenario from the dashboard or `orchestrator.py`. You do not hand-label rows — windows come from orchestrator timing in `metadata.json`.

17.1 Physical layer

Recorder writes `physical_raw.csv` and `physical_processed.csv` at `PHYS_SAMPLE_HZ` (20 Hz) with live emit at `LIVE_EMIT_HZ` (60 Hz) for the twin [3].

Source messages. `LOCAL_POSITION_NED`, `ATTITUDE`, `GLOBAL_POSITION_INT`, `VFR_HUD`, `SYS_STATUS`, `SERVO_OUTPUT_RAW`, `POSITION_TARGET_LOCAL_NED`, `HEARTBEAT`.

17.2 Network layer

`tcpdump` capture toward the UAV is windowed into rates and MAVLink msgid histograms [20, 19]. Disable via UI toggle or `--no-network` if privileges are unavailable.

17.3 Labels

- `label_phase`: `normal_plan|attack`
- `label_class`: `benign|<attack_id>`
- Pre and post windows are **benign** (same normal plan).

See `datasets/DATA_DICTIONARY.md`.

17.4 File layout per run

```

datasets/runs/<scenario>/run_XX/
  physical_raw.csv
  physical_processed.csv
  network_raw.csv # if capture enabled
  network_processed.csv
  metadata.json # attack windows, seed, timing

```



Figure 17.1: Finder/terminal listing of a `gps_spoofing run_XX` folder

17.5 Physical columns (selected)

Attitude (`roll`, `pitch`, `yaw`), NED pose/velocity, global position, battery, servo/motor outputs, setpoint targets, armed/mode fields, plus derived `speed`, `tilt_mag`, `pos_err_z`.

17.6 Network columns (selected)

Windowed `pkt_rate`, `byte_rate`, direction counts, and MAVLink msgid counters (`gps_input_count`, `command_long_count`, ...) used as IDS signatures.

17.7 How to collect a clean set

```

# Defence OFF for signatures; network ON when possible
./run_dashboard.sh
# UI: Start sim →unchecked Defence →run benign + each Tier A attack
# Or CLI:
.venv/bin/python orchestrator.py --scenarios benign,gps_spoofing --runs 3

```

Warning: Close QGroundControl before capture so the physical recorder can bind UDP 14550.

17.8 Troubleshooting collection

- Empty physical CSV — SITL not up or 14550 conflict.
- Missing network CSV — `tcpdump` privilege / wrong `NET_IFACE`.
- All-benign labels during attack — orchestrator aborted early; read live log.

Chapter 18

Data Generation

Data *generation* is the experimental protocol that produces diverse, comparable runs for training and evaluation.

18.1 Shared mission protocol

All scenarios fly the same `MISSION_PLAN` from `config.py`. Attacks pause after a gate waypoint, inject for `ATTACK_DUR_S` (default 12s), then resume [24].

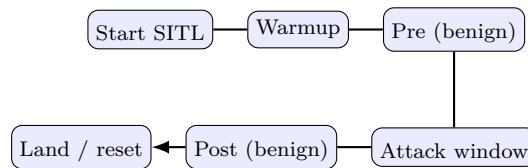


Figure 18.1: Shared data-generation timeline for every scenario.

18.2 Attack gates

`ATTACK_AFTER_WP` (default 4) delays the first eligible gate. `ATTACK_GATE_FRACTION` (default 0.5) randomly selects about half of eligible mid-mission waypoints as attack gates each run.

18.3 Commands

```
# List scenarios
.venv/bin/python orchestrator.py --list
# Matrix (example)
.venv/bin/python orchestrator.py --runs 5
# Quick smoke
RUN_DURATION_S=40 .venv/bin/python orchestrator.py \
  --scenarios benign,gps_spoofing --runs 1
```

Explanation: Dashboard **Run Full Pipeline** iterates Tier A scenarios; live-train can fine-tune models after runs.

18.4 Tier A vs Tier B

Tier A (core): benign, gps_spoofing, disarm_injection, mode_change_land, mission_injection, command_flood_dos, rc_override, param_injection.

Tier B: mode_change_rtl, heartbeat_spoof, takeoff_injection.

18.5 Reproducibility tips

- Keep Defence OFF while building the training corpus.
- Keep the same MISSION_PLAN; do not edit waypoints mid-study.
- Record `metadata.json` seeds when comparing papers tables.

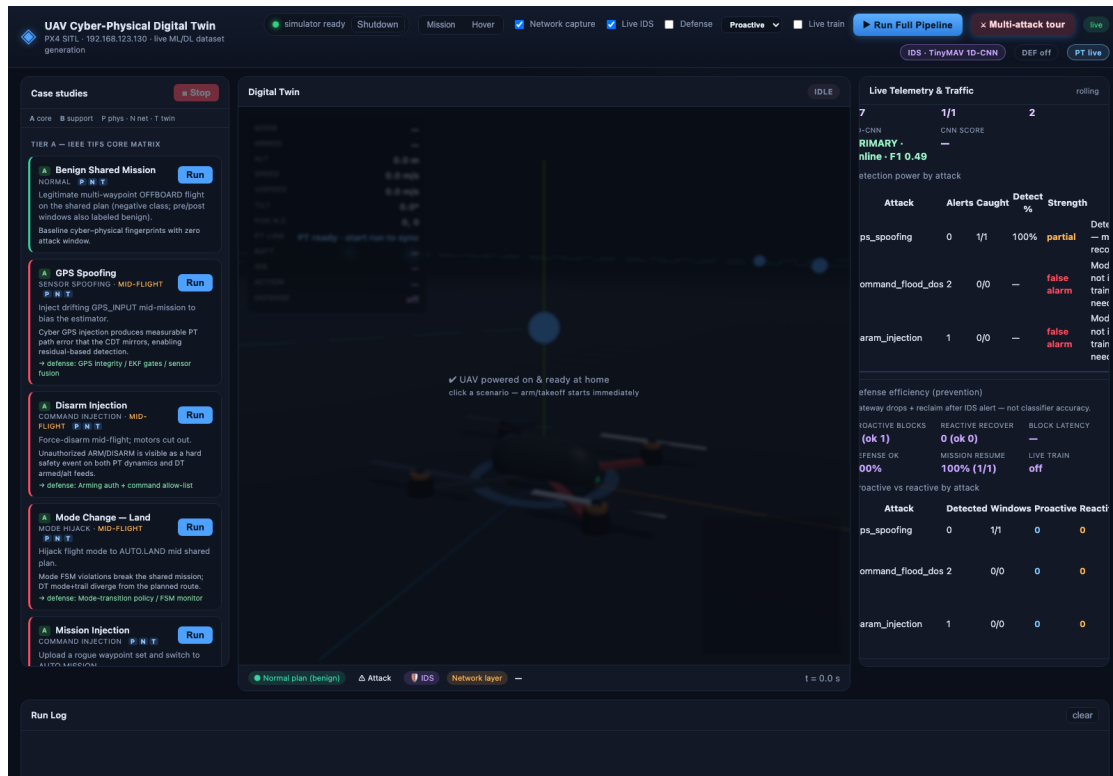


Figure 18.2: Dashboard pipeline progress across Tier A scenarios

Chapter 19

Data Preprocessing

Preprocessing converts raw recorder outputs into model-ready matrices.

19.1 Pipeline stages

1. Per-run raw $\rightarrow$ processed CSVs inside the recorder.
2. Optional merge via `build_dataset.py` into study-level tables.
3. Feature selection / drops in `ids/features.py`.
4. Sequence windowing for TinyMAV CNN (`ids/cnn_train.py`).
5. Scalers / encoders persisted inside LightGBM bundles.

Beginner Note: Processed CSVs already include derived kinematics (speed, tilt) and labels. IDS code still drops identity/absolute fields that would leak or hurt generalisation.

19.2 Columns dropped from models

Physical drops include timestamps, lat/lon, and meta label columns. Network drops include `t_rel`, `win_s`, and meta columns. Core retained fields live in `PHYSICAL_CORE` / `NETWORK_CORE`.

19.3 CNN sequences

TinyMAV stacks `DEFAULT_SEQ_LEN=8` consecutive network windows (channel $\times$ time) before convolution [14].

```
# Conceptual
X.shape # (N, C, T) with T=8 network windows
y_bin # 1 during attack windows
y_multi # benign | attack_id
```

19.4 Live vs offline normalisation

Offline training fits normalisation statistics into artefacts. Live scoring applies the same stats in `NetSeqCNNScorer` / `CascadeIDS`. Retrain after major protocol changes.

19.5 Practical commands

```
.venv/bin/python build_dataset.py  
.venv/bin/python -m ids cnn --epochs 12
```

Common Mistake: Training only on proactive-Defence ON runs yields almost no positive network signatures — the gateway already dropped them.

Chapter 20

Digital Twin Scripts

20.1 `orchestrator.py`

Purpose: end-to-end scenario execution.

Inputs: scenario id, run count, network flag, env timing overrides.

Outputs: `datasets/runs/<scenario>/run_XX/` + metadata.

Key flags: `--list`, `--scenarios`, `--runs`, `--no-network`.

20.2 `ssh_control.py`

Starts/stops SITL over SSH; `pt_status()`; `wait_ready()`. Uses `UAV_HOST` / `SSH_USER` from `config.py` (defaults `192.168.123.130` / `danish`).

20.3 `mav_common.py`

Shared MAVLink helpers: link via gateway, arm/mode/setpoint helpers, `update_vehicle_state` for DT pose, abort flags.

20.4 `config.py`

Central constants: ports, mission plan, attack gates, defence thresholds, `IDS_PRIMARY_MODEL`, `DEFENSE_MODE`.

20.5 `build_dataset.py`

Merges runs into aggregated CSVs and refreshes the data dictionary summary.

20.6 `run_dashboard.sh`

Primes sudo for tcpdump and launches Uvicorn on `127.0.0.1:8000`.

20.7 Essential launch

```
cd /Users/danishvasan/Downloads/uav-cyber-ml
./run_dashboard.sh
# open http://127.0.0.1:8000
```

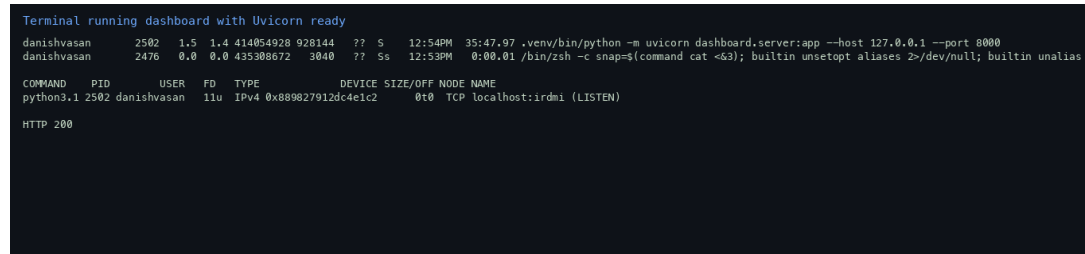
Explanation: Prefer the dashboard for demos; use orchestrator CLI for unattended matrices.

20.8 IDS CLI entrypoints

```
.venv/bin/python -m ids # LightGBM train
.venv/bin/python -m ids cnn # TinyMAV train
.venv/bin/python -m ids score # offline score helpers
```

20.9 Environment overrides (common)

```
export UAV_HOST=192.168.123.130
export MAV_GATEWAY=1
export DEFENSE_MODE=proactive
export GPS_SPOOF_DRIFT=3e-5
export NET_IFACE=en0
```



```
Terminal running dashboard with Uvicorn ready
danishvasan 2502 1.5 1.4 414054920 928144 ?? S 12:54PM 35:47.97 .venv/bin/python -m uvicorn dashboard.server:app --host 127.0.0.1 --port 8000
danishvasan 2476 0.0 0.0 435308672 3040 ?? Ss 12:53PM 0:00.01 /bin/zsh -c snap=s(command cat <63); builtin unsetopt aliases 2>/dev/null; builtin unalias

COMMAND PID USER FD TYPE DEVICE SIZE/OFF NODE NAME
python3.1 2502 danishvasan 11u IPv4 0x889027912dc4e1c2 0t0 TCP localhost:1rdmi (LISTEN)

HTTP 200
```

Figure 20.1: Terminal running run_dashboard.sh with Uvicorn ready message

Chapter 21

Digital Twin Services (Detailed)

Each service below maps to code under `dashboard/`, `recorders/`, and `ids/`. Insert real screenshots where indicated.

Beginner Note: You do not start these services individually. Launching `./run_dashboard.sh` brings up FastAPI, which wires recorders, IDS, and the gateway as scenarios run.

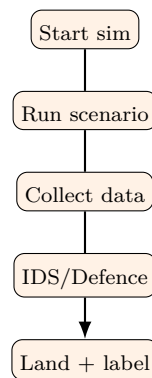


Figure 21.1: Digital Twin operational workflow.

21.1 Real-time telemetry

Purpose. Mirror PX4 pose/attitude in the browser.

Inputs. MAVLink LOCAL_POSITION_NED, ATTITUDE on UDP 14550.

Outputs. WebSocket `phys` samples; Three.js pose [13].

Rate. ≈ 60 Hz live emit (`LIVE_EMIT_HZ`).

Implementation. `physical_recorder.py` $\rightarrow$ bus $\rightarrow$ `app.js`.

Verify. PT link LIVE; altitude HUD changes when Gazebo vehicle climbs.

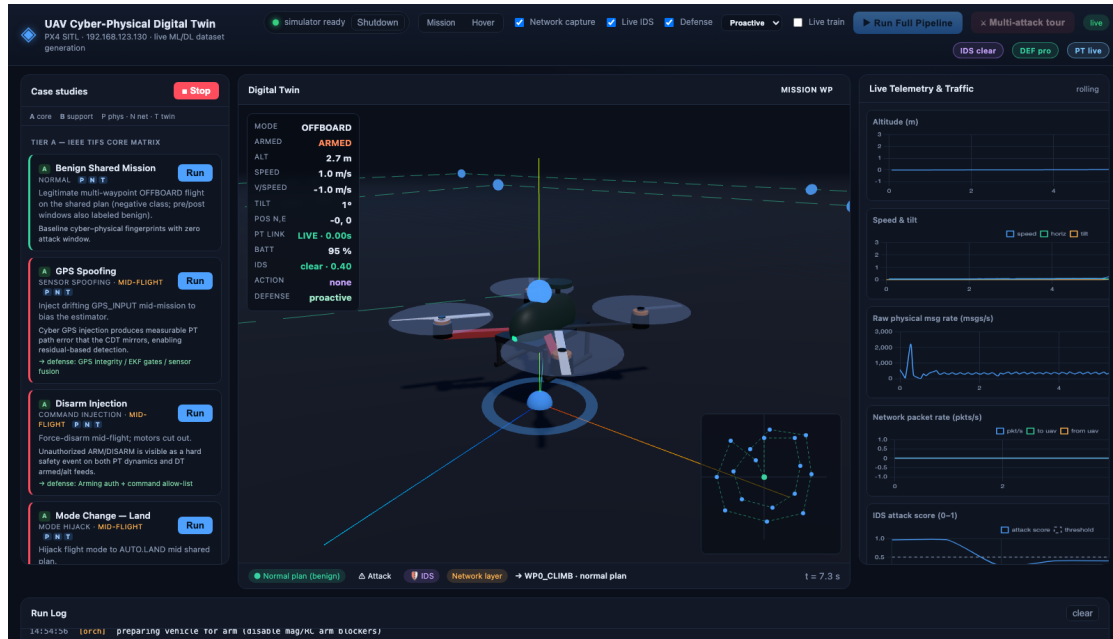


Figure 21.2: Digital Twin with PT link LIVE and altitude HUD

21.2 Live mission monitoring

Purpose. Show orchestrator phase and waypoint progress.

Inputs. Orchestrator hooks / state events (mission_wp, phase).

Outputs. Banners, waypoint labels, timeline markers.

Implementation. orchestrator.py hooks published via dashboard/server.py WebSocket bus.

Verify. During a run, waypoint ids from MISSION_PLAN advance.

21.3 UAV state monitoring

Purpose. Expose armed/mode/altitude/speed/battery.

Inputs. HEARTBEAT, SYS_STATUS, VFR_HUD, derived kinematics.

Outputs. HUD fields and /api/state snapshot.

Verify. Arming in Gazebo flips Armed indicator in the DT.

21.4 Sensor synchronisation

Purpose. Align physical and network modalities on one timeline.

Mechanism. Shared t_rel and attack windows from metadata.json; live bridge aggregates physical windows to match network window cadence.

Verify. Dataset explorer shades the same attack interval on phys and net charts.

21.5 Data collection module

Purpose. Start/stop recorders for each scenario.

Trigger. Scenario enters recording after SITL ready.

Outputs. raw/processed CSVs per modality.

Verify. New `run_XX` directory appears under `datasets/runs/`.

21.6 Data storage module

Purpose. Persist runs for training and replay.

Layout. `datasets/runs/<scenario>/run_XX/` plus merged tables from `build_dataset.py`.

Verify. `metadata.json` contains timing, seed, and attack gates.

21.7 Data labelling module

Purpose. Attach supervised labels without manual row edits.

Rules. Pre/post = benign; only attack windows = attack class.

Implementation. Orchestrator timing + recorder label columns.

Verify. `label_class` changes only inside the attack interval.

21.8 Data preprocessing module

Purpose. Derive features and prepare IDS tensors.

Implementation. Recorder processed columns + `ids/features.py` + CNN sequence builder.

Verify. Training logs report sequence counts and attack rate.

21.9 AI prediction module

Purpose. Online inference during flight.

Implementation. `ids/live_bridge.py` with TinyMAV (`IDS_PRIMARY_MODEL=cnn1d`) and optional LightGBM cascade.

Outputs. Attack score, predicted class, latency.

Verify. Score rises during Defence OFF attack windows after training.

21.10 Live IDS module

Purpose. Operator-facing detection UI.

Outputs. Score HUD, alerts, metrics table, unprotected pill when artefacts missing.

APIs. GET/POST `/api/ids`.

Verify. After `python -m ids cnn`, unprotected pill clears.

21.11 Live defence module

Purpose. Prevent or recover from injections.

Modes. proactive / hybrid / reactive (and aliases) via gateway + reclaim.

Implementation. `ids/mav_gateway.py`, `ids/defense.py`.

Verify. Proactive GPS spoof increments `proactive_blocks`.

21.12 Dashboard module

Purpose. Human control plane for the cyber-range.

Implementation. FastAPI REST + WebSocket [11, 12]; static Three.js client.

URL. `http://127.0.0.1:8000`

Verify. Hard refresh after code pulls; scenario buttons respond.

21.13 Logging module

Purpose. Stream orchestrator/pilot/attack/IDS messages.

Outputs. Tagged live log pane; useful for playbook troubleshooting.

Verify. Attack start lines appear when a gate fires.

21.14 Replay / dataset explorer

Purpose. Inspect historical CSVs without re-flying.

Implementation. `dashboard/datasets.py` + explorer UI.

Verify. Selecting a past run plots raw/processed series with shaded attacks.

21.15 Alert module

Purpose. Surface IDS hits and proactive blocks.

Outputs. Banners / pills; gateway events on the bus.

Verify. PROACTIVE BLOCK lines during defended `gps_spoofing`.

21.16 Event timeline

Canonical sequence. `sitl_starting` → `recording` → `mission_wp` → `attack` → `landing` → `done`.

Verify. Timeline widgets and logs follow this order.

21.17 Mission replay

Purpose. Compare twin trail vs planned waypoints after the fact.

Inputs. Processed physical pose + `MISSION_PLAN`.

Verify. Benign trails follow the shared route; spoofed runs diverge when Defence OFF.

21.18 Performance analytics

Purpose. Summarise detection/mitigation quality.

Metrics. Precision/recall/F1/FPR, detection delay, mitigation delay, defence success, mission resume, proactive vs reactive counts (see PAPER_PROTOCOL in config.py).

Verify. Analytics panel populates after defended runs.

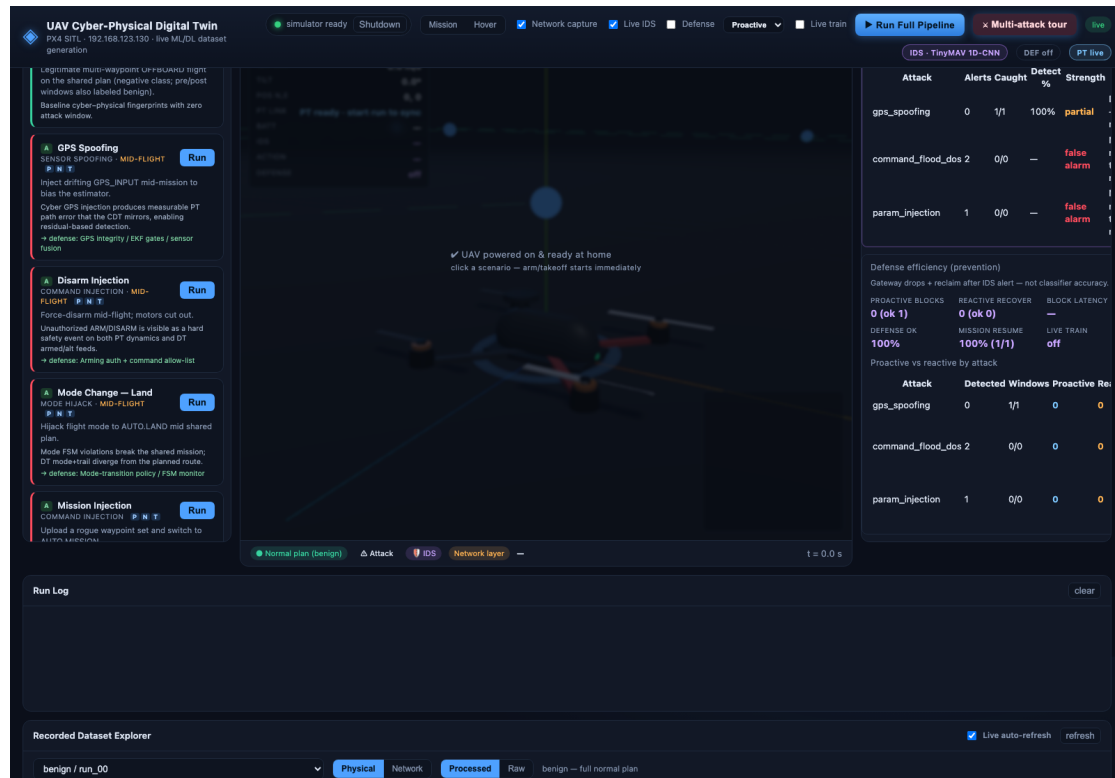


Figure 21.3: Performance analytics panel with proactive block counts

Chapter 22

Cyber-Attack Scenarios (Complete Playbooks)

All scenarios share one OFFBOARD mission from `config.MISSION_PLAN`. Attacks inject after gate waypoints. Code: `attacks/suite.py`; pilot: `attacks/benign.py`.

Beginner Note: Always capture Defence OFF runs for training labels, then repeat with Defence ON to evaluate mitigation.

22.1 Shared protocol

1. Start sim (SSH → PX4+Gazebo).
2. Run scenario from UI or orchestrator.
3. Pre (benign labels) → attack window(s) → post (benign).
4. Land / reset; inspect CSVs and logs.

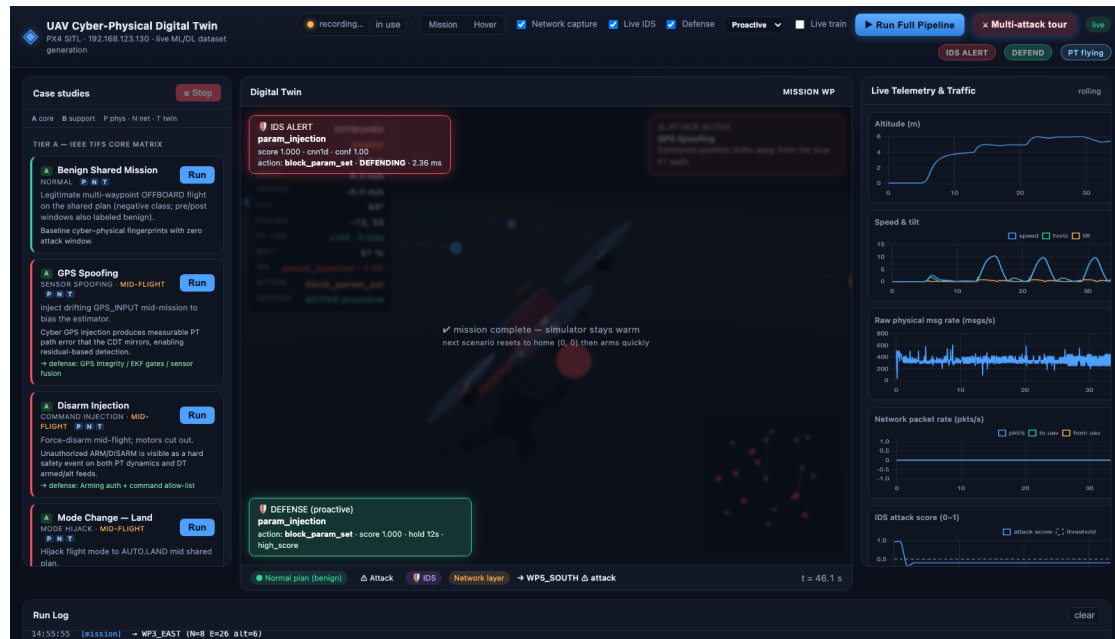


Figure 22.1: Mission route with attack gate highlighted

22.1.1 C

common commands

```
./run_dashboard.sh
# UI: Start sim →select scenario →Defence OFF →Run
.venv/bin/python orchestrator.py --scenarios gps_spoofing --runs 1
```

22.2 Benign

Intro. Negative class: shared mission with no injector.

Theory. Establishes baseline kinematics and network mix.

Expected PT. Smooth Iris flight along plan.

Expected DT. Twin tracks PT; no attack banner.

Expected IDS. Low scores after training.

Defence. N/A (nothing to block).

Recovery. Normal land.

22.3 GPS spoofing

Intro. Drift GPS_INPUT (msgid 232) to bias navigation.

Theory. Compromised GCS feeds false geo to EKF [24].

Workflow. Gate → attacker sysid 250 sends drifting GPS_INPUT.

```
m = _attacker() # sysid 250 via gateway
_attacker_ready(m)
while time.time() - t0 < duration_s:
```

```
if _ips_aborted(logf, "gps_spoofing"):
    break
m.conn.mav.gps_input_send(... drifting lat/lon ...)
```

Explanation: Proactive mode should drop msgid 232 and increment proactive_blocks.

Expected PT (Defence OFF). Position wander / unstable hold.

Expected DT. Trail divergence vs plan.

Expected IDS. Score↑; class `gps_spoofing` when trained.

Defence ON / proactive. Packets dropped pre-PX4; vehicle stays on plan.

Recovery. Abort injector; clear nav bias; resume plan / land.

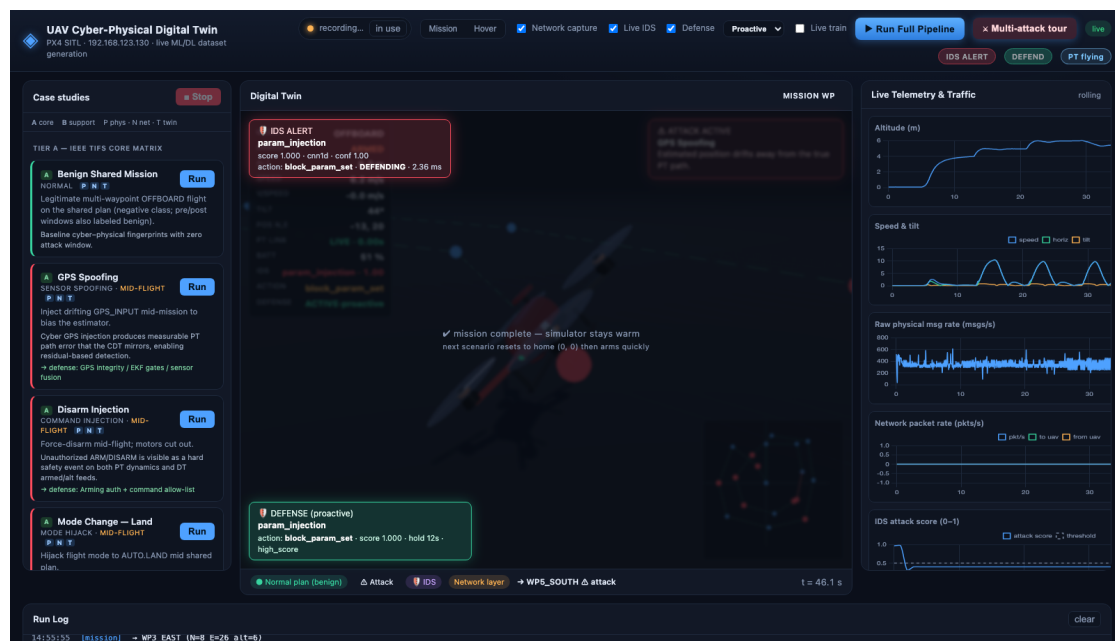


Figure 22.2: GPS spoof — DT trail divergence

22.4 Disarm injection

Intro. Force-disarm `COMMAND_LONG` mid-air.

Theory. Spoofed GCS kills motors.

Expected PT (OFF). Armed flag drops; vehicle falls in Gazebo.

Expected DT. Armed HUD false; altitude collapse.

Expected IDS. High score on command signatures / physical collapse.

Defence. Abort flood; reclaim / keep armed under IPS.

Recovery. Re-arm only if safe; usually reset SITL after crash demos.

22.5 Mode change LAND / RTL

Intro. Spoofed `SET_MODE` to `AUTO.LAND` or `RTL`.

Expected PT (OFF). Mission aborts into land/RTL.

Expected DT. Mode HUD changes; path toward land/home.

IDS. Mode-change signatures + path residual.

Defence. Drop SET_MODE from attacker; reclaim OFFBOARD.

Recovery. Force OFFBOARD and resume setpoints / land cleanly.

22.6 Mission injection

Intro. Rogue mission items / counts from attacker.

Expected PT/DT (OFF). Route changes toward injected WPs.

IDS. Mission msgid counters + path residual.

Defence. Drop mission msgids; restore plan via pilot.

Recovery. Clear rogue mission; continue shared plan.

22.7 Command flood DoS

Intro. COMMAND_LONG spam to saturate link/autopilot.

Expected PT/DT (OFF). Lag, twin stutter, mode/arm glitches.

IDS. Network rate spike; command_long_count↑.

Defence. Drop flood; reclaim control.

Recovery. Stop injector; wait settle; resume.

22.8 RC override

Intro. MANUAL_CONTROL frames hijack sticks.

Expected PT/DT (OFF). Abrupt kinematics / tilt.

IDS. manual_control_count↑ + physical jerk.

Defence. Drop MANUAL_CONTROL; hold OFFBOARD.

Recovery. Abort override; stabilise; resume plan.

22.9 Param injection

Intro. PARAM_SET to alter controller / failsafes.

Expected PT. Often subtle physically; strong network signature.

IDS. param_set_count↑ (network-first).

Defence. Drop PARAM_SET from attacker.

Recovery. Reset SITL if parameters corrupted.

22.10 Tier B playbooks

Heartbeat spoof. Fake HEARTBEAT identity noise — see CASE_STUDIES.md.

Takeoff injection. Unauthorized arm+takeoff — dangerous on real hardware; safe in SITL; defence should abort.

Mode change RTL. Like LAND but RTL mode.

22.11 Multi-attack tour

One flight, multiple Tier A gates via dashboard multi-tour / orchestrator schedule. Use for stress demos; prefer single-attack runs for clean training labels.

22.12 Troubleshooting playbooks

- Zero proactive counts: Defence ON, mode Proactive/Hybrid, model loaded, observe/grace window not aborting before first packet.
- No physical effect with Defence OFF: attacker link not through gateway; check MAV_GATEWAY=1 and UAV reachability.
- IDS silent: train CNN; confirm artefacts; Defence OFF for signature check.

Warning: Never run these injectors against a real aircraft. SITL only.

Chapter 23

Phase 2 Summary

Phase 2 connects the Physical Twin to the operational Digital Twin in `uav-cyber-ml`: FastAPI + Three.js at `http://127.0.0.1:8000`, recorders, labelled runs under `datasets/runs/`, and attack playbooks.

23.1 You should now be able to

1. Install the Python venv and launch `./run_dashboard.sh`.
2. Start SITL from the UI and see PT link LIVE with a moving 3D twin.
3. Run benign and Tier A attack scenarios with shared `MISSION_PLAN`.
4. Locate per-run CSVs and `metadata.json` attack windows.

23.2 Exit checklist

- `curl http://127.0.0.1:8000/api/pt` eventually reports healthy processes.
- At least one benign and one attack run exist under `datasets/runs/`.
- Defence OFF captures show clear attack effects for later IDS training.

Part III

Phase 3 — AI Intrusion Detection System (IDS)

Chapter 24

IDS Overview

24.1 Purpose

Detect MAVLink cyber attacks during shared missions and optionally prevent or recover via defence modes [24].

Beginner Note: IDS here means both detection (scores/alerts) and optional prevention (gateway drops) / recovery (reclaim). Without trained artefacts the lab forces Defence OFF so you can still collect signatures.

24.2 Architecture

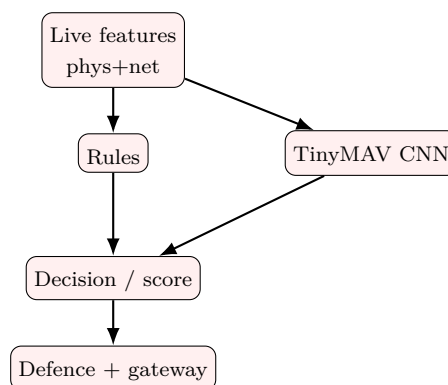


Figure 24.1: AI IDS pipeline from features to defence.

24.3 Primary vs secondary models

IDS_PRIMARY_MODEL defaults to `cnn1d` (TinyMAV). LightGBM physical/network/fusion bundles remain available as fallback/cascade (`fusion`) [14, 16].

24.4 Unprotected mode

If `ids/artifacts/` lacks a trained model, Defence is forced off so capture runs show full attack effects.

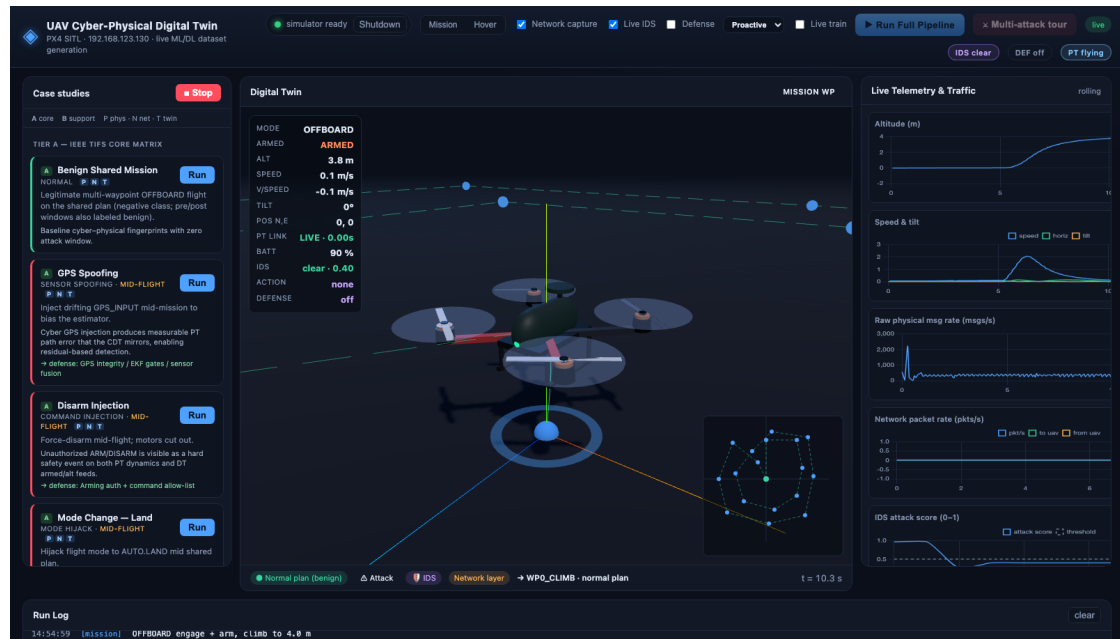


Figure 24.2: Dashboard showing IDS unprotected pill

24.5 Key files

ids/live_bridge.py, ids/cnn_model.py, ids/train.py, ids/mav_gateway.py, ids/defense.py, ids/rules.py.

Chapter 25

IDS Dataset

25.1 Source

Per-run folders under `datasets/runs/` plus optional merged tables from `build_dataset.py`.
Dictionary: `datasets/DATA_DICTIONARY.md`.

25.2 Label semantics

Pre/post windows are benign even on attack scenarios. Only orchestrator attack intervals carry positive labels. This matches the shared-mission protocol.

25.3 Class inventory (Tier A)

benign, gps_spoofing, disarm_injection, mode_change_land, mission_injection, command_flood_dos, rc_override, param_injection.

25.4 Files the trainers read

Trainers prefer `network_processed.csv` / `physical_processed.csv` with columns listed in the data dictionary. Missing network files degrade CNN training (TinyMAV is network-window based).

25.5 Balance notes

GPS spoof windows may be shorter/rarer depending on gates; collect enough runs per class (≥ 6 –10 recommended for papers).

25.6 Loading for training

```
.venv/bin/python -m ids # uses processed runs  
.venv/bin/python -m ids cnn
```

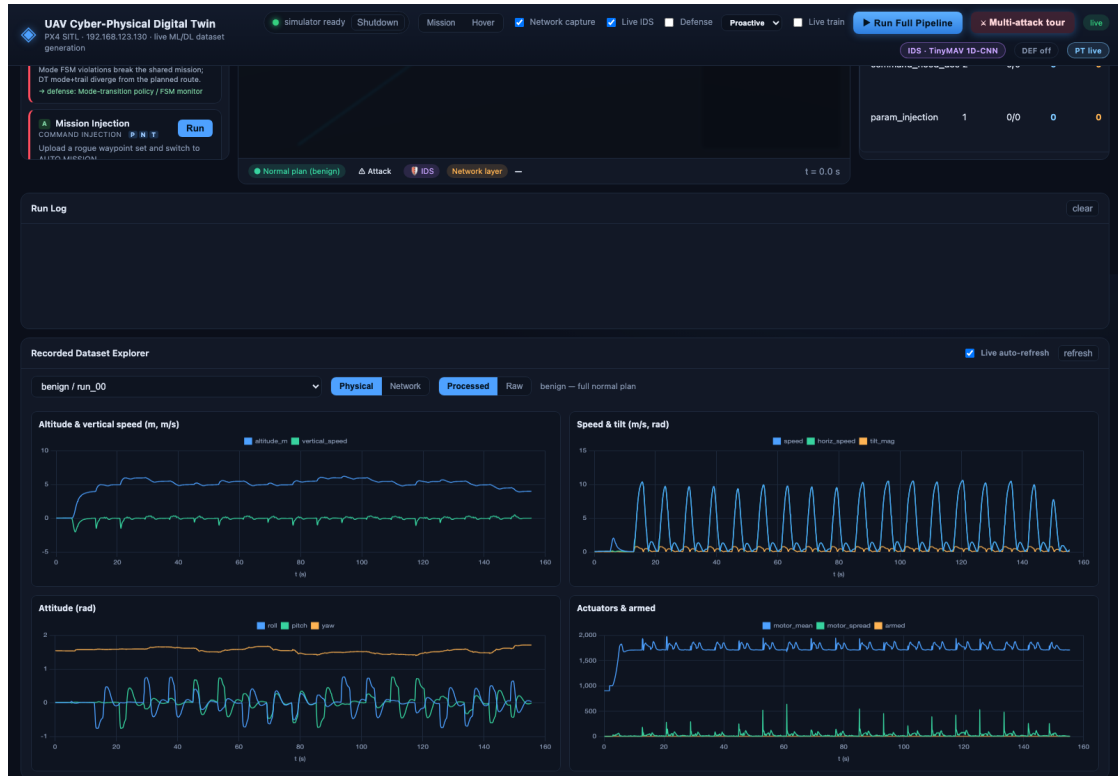


Figure 25.1: Dataset explorer showing a labelled physical CSV

25.7 Quality checks

- Attack rows exist only inside metadata windows.
- Benign scenario has `label_binary=0` throughout.
- Network capture present for CNN studies.

Best Practice: Keep a held-out run per scenario for sanity checks after every retrain.

Chapter 26

Feature Engineering

26.1 Physical features

Core attitude, rates, NED pose/velocity, motors, setpoints, armed/mode, plus derived speed/tilt/-position error (PHYSICAL_CORE in `ids/features.py`).

26.2 Network features

Windowed packet/byte rates, direction counts, unique msgids/sysids, and per-message counters (heartbeat, command_long, param_set, mission_item, manual_control, gps_input, set_mode) — NETWORK_CORE.

26.3 What is dropped

Timestamps, lat/lon, and label/meta columns are excluded from model inputs to avoid leakage and non-causal shortcuts.

26.4 Fusion

Cascade/fusion models concatenate aligned physical-window aggregates with network windows inside the live bridge.

26.5 Why network helps

Several attacks (param injection, early command floods) are network-first; physical residuals appear later or weakly.

26.6 Live normalisation

```
# live_bridge.normalize_network_window(d)
# maps sniffer fields onto the training schema
```

Common Mistake: Renaming recorder columns without retraining silently zeros features.

26.7 Inspecting a feature row

Open a processed CSV in the dataset explorer and confirm counters spike only in shaded attack windows for network-heavy attacks.

Chapter 27

Model Architecture

27.1 TinyMAV 1D-CNN (primary)

Two Conv1d layers + BatchNorm + ReLU + adaptive average pool, then dual heads: binary attack logit and multiclass logits [14].

```
# ids/cnn_model.py (conceptual)
backbone = Conv1d -> BN -> ReLU -> Conv1d -> BN -> ReLU -> AdaptiveAvgPool1d(1)
binary_head = Linear(hid*2, 1)
multi_head = Linear(hid*2, n_classes)
# Input shape: (batch, channels=NETWORK_CORE, time=8)
```

27.2 Why TinyMAV

Edge/GCS friendly latency; strong on cyber MAVLink patterns; small artefact size (cnn_mav1d.pt + cnn_mav1d_meta.json).

27.3 LightGBM cascade (secondary)

Modality bundles: physical, network, fusion (ids/train.py, ids/live_scorer.py) [16, 15]. Useful when CNN artefacts are missing or for ablation studies.

27.4 Rules layer

ids/rules.py provides critical heuristics that can reinforce class labels when model confidence is borderline.

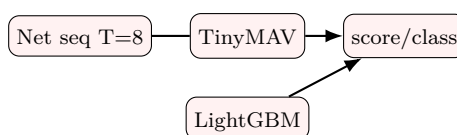


Figure 27.1: Primary CNN with optional LightGBM assist.

Chapter 28

Model Training

28.1 Prerequisites

At least one benign run and several attack runs with Defence OFF and (preferably) network capture enabled.

28.2 Offline training

```
cd /Users/danishvasan/Downloads/uav-cyber-ml
.venv/bin/python -m ids # LightGBM modalities
.venv/bin/python -m ids cnn --epochs 12
```

28.3 Dashboard / API training

```
curl -X POST http://127.0.0.1:8000/api/train \
-H 'Content-Type: application/json' \
-d '{"live_train":true,"retrain_now":true}'
```

Explanation: Offline trainers write artefacts; API retrain works without a new SITL flight.

28.4 Artefacts

- ids/artifacts/cnn_mav1d.pt, cnn_mav1d_meta.json
- LightGBM *_bundle.json / metrics JSON files
- live_train_state.json for UI status

28.5 Validation

Holdouts/group folds; metrics include F1, recall, AUC, latency [15]. Inspect `cnn_mav1d_report.txt` after CNN training.

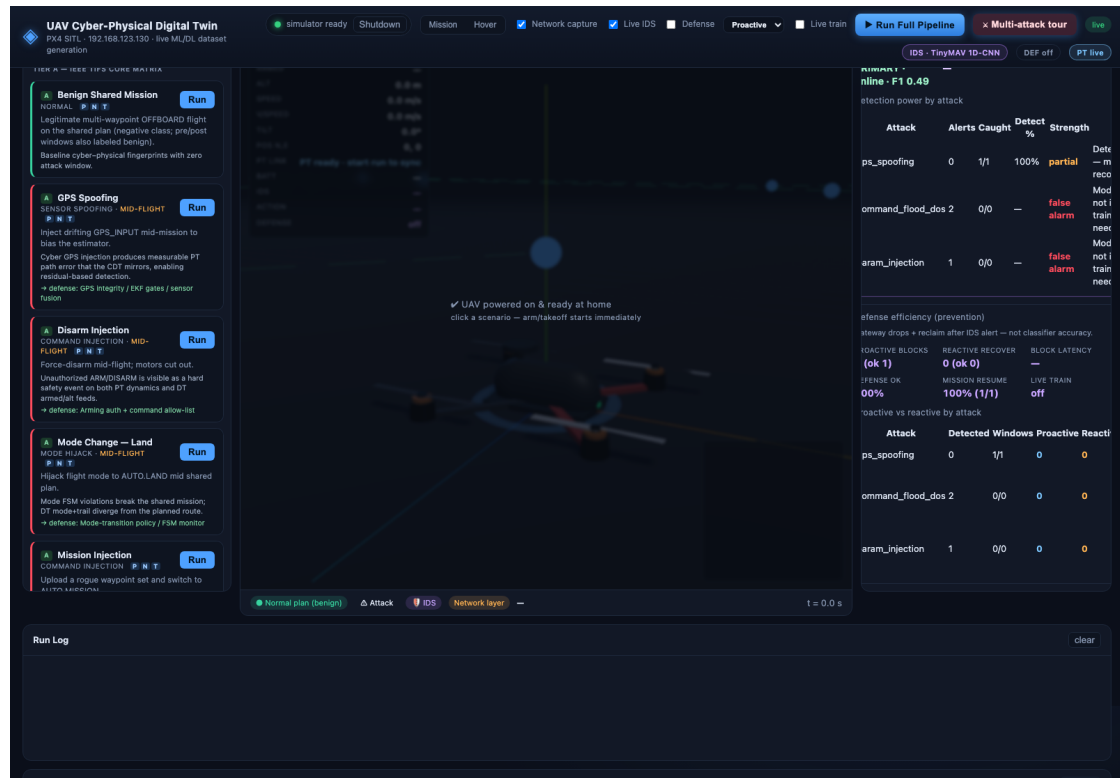


Figure 28.1: Training log showing cnn1d live train OK

28.6 Troubleshooting training

- Too few sequences — collect more network-enabled runs.
- Single-class early models — add diverse Tier A attacks.
- Torch missing — reinstall from `requirements.txt`.

Chapter 29

IDS Modes — Reactive, Proactive, Hybrid

Defence mode is selected in the UI or via `DEFENSE_MODE` / `POST /api/ids`. Gateway behaviour is implemented in `ids/mav_gateway.py`.

29.1 Reactive

Forward all traffic; after detect, reclaim OFFBOARD with defender sysid. Good for studying recovery; physical effect occurs first.

29.2 Proactive

Gateway drops dangerous attacker msgids before PX4. Best prevention; attack may be invisible on the vehicle and twin.

29.3 Hybrid

Short observable/signature grace, then drops (+ reclaim for some classes). Best for demos/papers that need both evidence and mitigation.

29.4 Soft / prevent aliases

`soft` $\approx$ short reactive reclaim; `prevent` aliases hybrid for older UI/scripts.

Mode	Gateway	Reclaim	Typical DT symptom
Reactive	forward	yes	visible then recover
Proactive	drop	minimal	mostly prevented
Hybrid	delayed drop	selective	short then stable

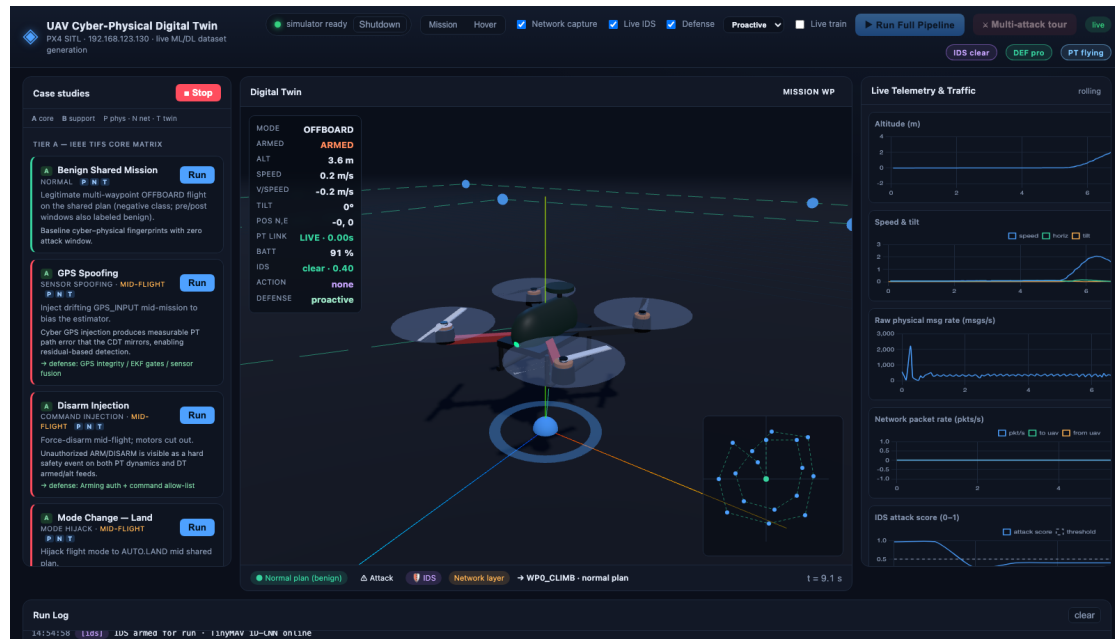


Figure 29.1: Defence mode dropdown set to Proactive with Defence ON

Warning: Without trained artefacts, mode selection has no effect — Defence stays UN-PROTECTED.

29.5 Engagement thresholds

DEFENSE_ENGAGE_SCORE (default 0.72), DEFENSE_SIGNATURE_GRACE_S, DEFENSE_PREVENT_HOLD_S, DEFENSE_TRUST_MODEL control when reclaim/prevention engages.

Chapter 30

Attack Detection Behaviour

30.1 Live scoring path

Network windows (and physical aggregates) enter **LiveIDSEngine**. TinyMAV updates a rolling length-8 buffer; when ready it emits score/class/latency.

30.2 What “detection” looks like

- Score HUD rises above engage threshold.
- Alert banner names a class (e.g. `gps_spoofing`).
- Metrics table updates detection delay once ground-truth window known.

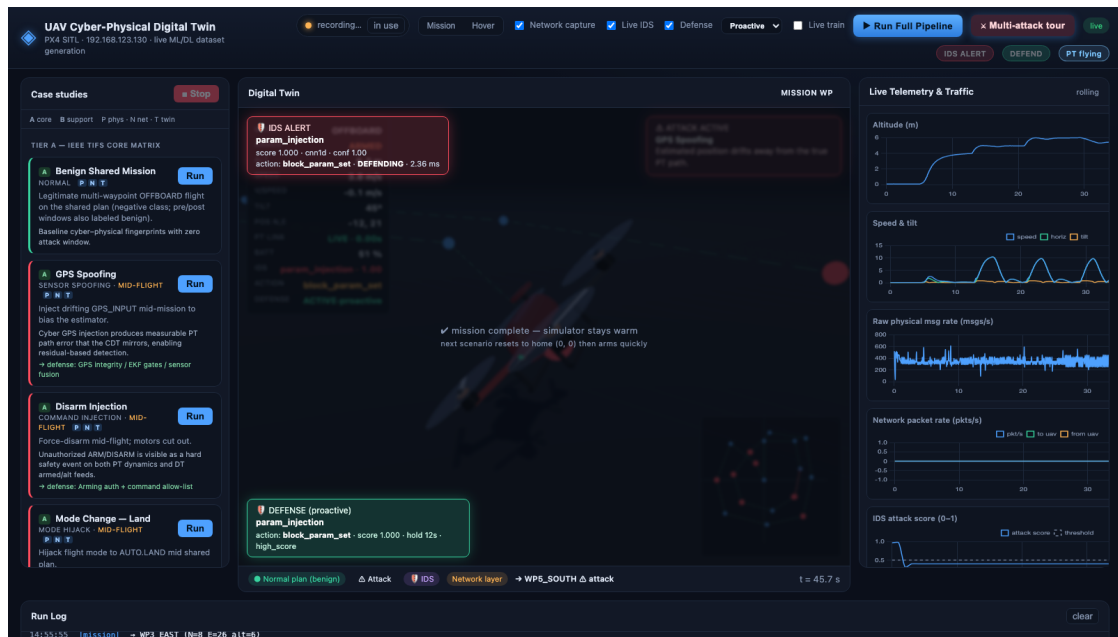


Figure 30.1: Attack detection logs and IDS alert banner

30.3 Per-attack cues

Attack	Strong cues
gps_spoofing	gps_input_count; trail divergence
disarm_injection	command_long; armed drop
mode_change_*	set_mode_count; mode HUD
mission_injection	mission_* counters; route residual
command_flood_dos	pkt/command rates
rc_override	manual_control_count; jerk
param_injection	param_set_count

30.4 False positives / negatives

FP: noisy Wi-Fi captures, QGC contention.

FN: proactive-only corpus, too few sequences, wrong schema mapping.

30.5 How to validate detection without defence

Defence OFF → run attack → confirm banner + CSV labels → only then enable Defence for mitigation tests.

Chapter 31

Live Defence

31.1 Enablement

Train models, ensure artefacts exist, toggle Defence ON, pick mode (proactive recommended for prevention demos).

31.2 Proactive path

Lab TX → 127.0.0.1:19550 → filter → UAV:14550. Dangerous msgids from attacker sysid 250 are dropped.

31.3 Reactive path

Packets reach PX4; IDS detects; defender reclaim restores OFFBOARD control and aborts injectors via `defense_should_abort()`.

31.4 Hybrid path

Allow a short signature grace (`DEFENSE_SIGNATURE_GRACE_S`), then engage drops and/or reclaim for the prevent hold duration.

31.5 Operator checklist

```
curl -s http://127.0.0.1:8000/api/ids | python3 -m json.tool | head -40
# Expect model ready, defense enabled, mode set
```

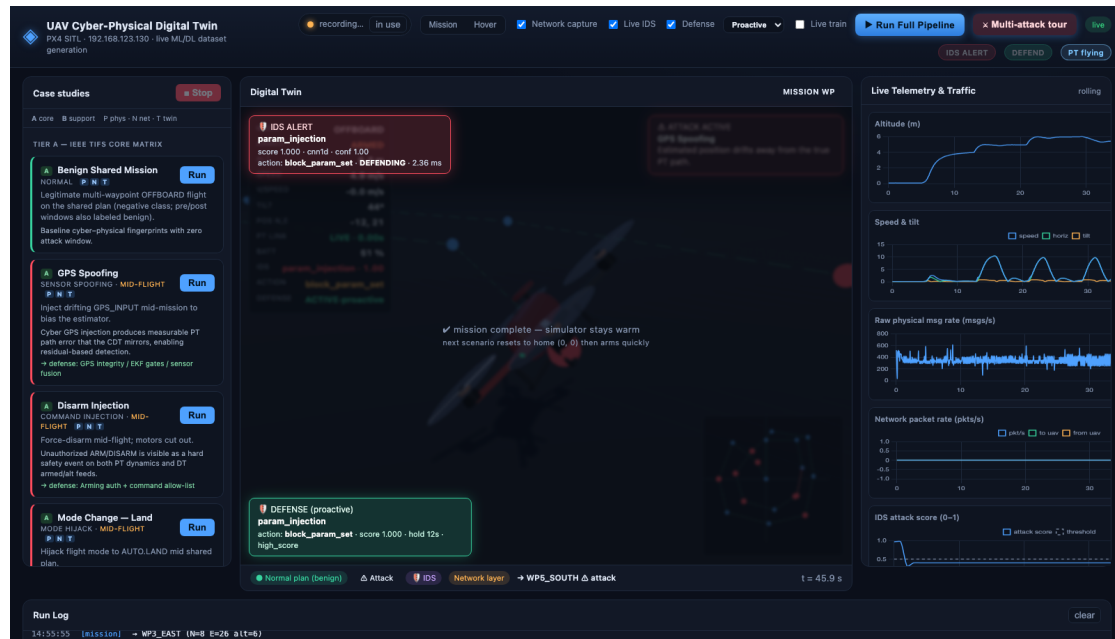


Figure 31.1: PROACTIVE BLOCK log with latency

31.6 Success criteria

- Proactive: blocks>0; mission continues near plan.
- Reactive: visible glitch then recover; mission_resume_ok.
- Hybrid: short signature then stabilise.

31.7 Abort coupling

Attack loops poll `_ips_aborted`; when IPS engages they stop injecting so the pilot can resume the shared plan.

Important: Trusted sysids always pass. Do not reuse attacker sysid 250 for the benign pilot.

Chapter 32

End-to-End System Architecture

32.1 Three planes

Physical Twin: Ubuntu UAV @ 192.168.123.130 — PX4 + Gazebo.

Digital Twin host: FastAPI/Three.js/recorders/IDS/gateway.

Human: browser at localhost:8000 + optional QGC (offline during capture).

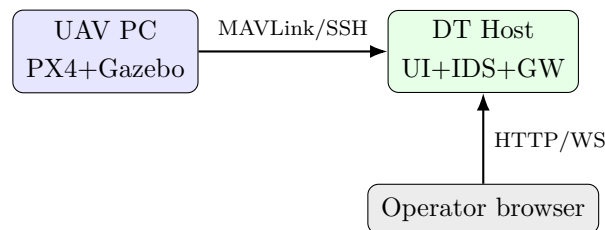


Figure 32.1: End-to-end actors in the cyber-range.

32.2 Sysid map

254 recorder, 252 controller/pilot, 250 attacker, 249 defender, PX4 1.

32.3 Process map on DT host

Uvicorn (dashboard), optional tcpdump, proactive gateway thread, recorder threads during runs, Torch/LightGBM scorers inside the live bridge.

32.4 Failure domains

Network partition, SSH key loss, 14550 conflict, missing artefacts, gateway bypass (`MAV_GATEWAY=0`) — each breaks a different plane.

32.5 Configuration single source

Prefer changing behaviour via `config.py` / environment variables rather than hardcoding ports in scripts.

Chapter 33

Performance Evaluation

33.1 Metrics tracked

Precision, recall, F1, FPR, mean detection delay, mean mitigation delay, defence success rate, mission resume, proactive blocks vs reactive recovers (`config.PAPER_PROTOCOL`).

33.2 How to gather

1. Build Defence OFF corpus and train.
2. Re-run Tier A with Defence ON (chosen mode).
3. Export/inspect analytics panel and metrics JSON under `ids/artifacts/`.

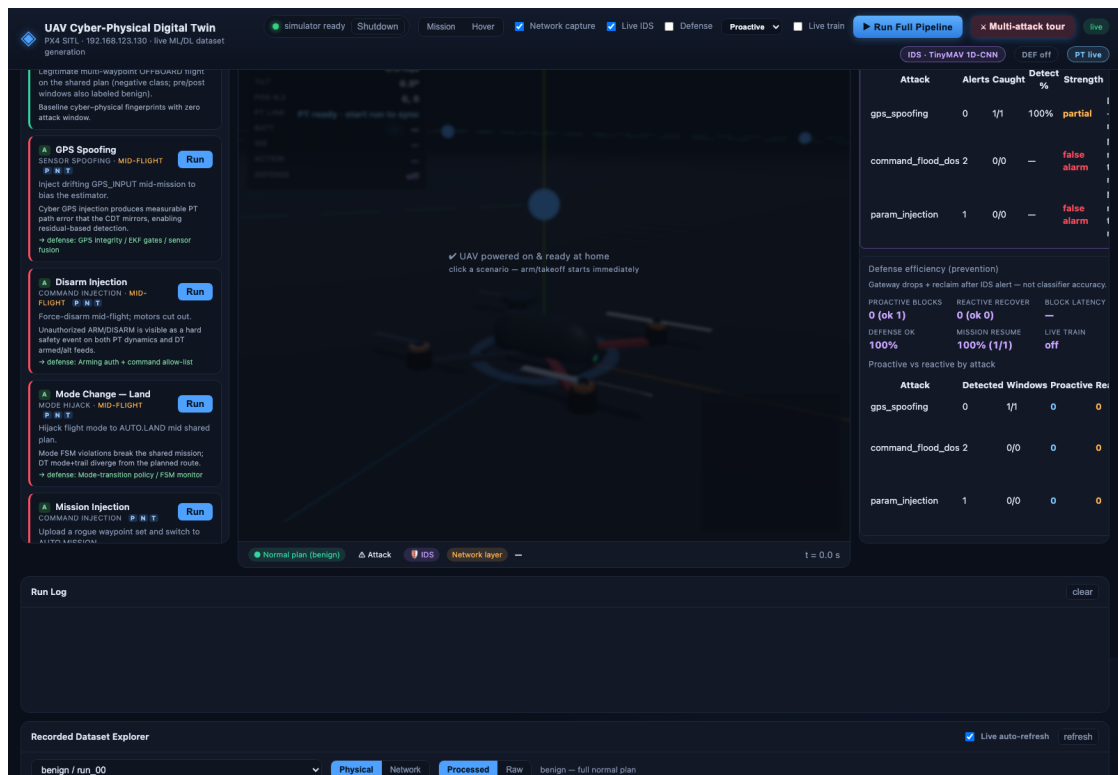


Figure 33.1: Performance analytics panel after a defended run

33.3 Suggested experimental matrix

≥ 10 runs/scenario; report mode; network on; gate fraction 0.5; separate detection vs mitigation tables.

33.4 Reporting tips

Separate tables for detection (OFF) and mitigation (ON). State gateway mode, run counts, and gate fraction. Do not pool proactive-blocked runs into detector training without noting distribution shift.

33.5 Latency

TinyMAV inference latency is logged per decision; proactive drops add microseconds of gateway filtering — far cheaper than post-crash recovery.

33.6 Where numbers live

Dashboard performance analytics; `ids/artifacts/*metrics*.json`; per-run `metadata.json` timing fields.

Chapter 34

Phase 3 Summary

Phase 3 adds the AI IDS: TinyMAV 1D-CNN (primary) and LightGBM modalities, live scoring, and proactive / hybrid / reactive defence through the MAV gateway on `127.0.0.1:19550`.

34.1 You should now be able to

1. Train models with `python -m ids` / `python -m ids cnn`.
2. Enable Defence and select proactive/hybrid/reactive modes.
3. Interpret score HUD, proactive block counts, and recovery banners.
4. Reproduce detection vs prevention behaviour per attack playbook.

34.2 Exit checklist

- Artefacts exist under `ids/artifacts/` (CNN weights + meta).
- Dashboard IDS pill is no longer forced UNPROTECTED.
- A defended GPS-spoof run shows gateway drops or reclaim (mode-dependent).

Chapter 35

Conclusion

This handbook walked from a clean Ubuntu UAV workstation through a MAVLink-centric Digital Twin to a live AI intrusion detection and defence stack. The reproducible artefact is the `uav-cyber-m1` repository together with passwordless SSH to the Physical Twin at `192.168.123.130`.

35.1 What the laboratory demonstrates

- Shared-mission labelling so pre/post flight remains benign.
- Dual-modality datasets (physical + network) aligned on one timeline.
- TinyMAV 1D-CNN online detection with optional LightGBM fusion.
- True pre-PX4 prevention via the proactive gateway when Defence is ON.

35.2 Operational takeaway

Treat ROS 2 as educational scaffolding. Day-to-day cyber-range work is: PX4+Gazebo SITL on the UAV PC, dashboard on localhost:8000, pymavlink through gateway :19550, and IDS artefacts under `ids/artifacts/`.

Important: Always capture Defence OFF runs before claiming detector performance; prevention hides the very signatures the model must learn.

Chapter 36

Future Work

36.1 Near-term lab extensions

- Increase per-scenario run counts for stronger journal tables (≥ 10).
- Automate paper metric export from dashboard performance analytics.
- Harden network-capture privilege setup across fresh macOS versions.

36.2 Research directions

- Multi-UAV / swarm twins with per-vehicle gateway policies.
- Hardware-in-the-loop (HITL) validation of the same IDS artefacts.
- Adversarial MAVLink crafting against TinyMAV and LightGBM.
- Optional ROS 2/DDS attack surfaces as a parallel educational track.

36.3 Engineering debt worth addressing

Documented but still easy to misconfigure: QGC vs 14550 peer sticky behaviour, tcpdump interface autodetection, and early single-class CNN thresholds.

Tip: Prefer extending `CASE_STUDIES.md` and this manual together so new attacks ship with a playbook on day one.

Bibliography

- [1] Dronecode Foundation, *PX4 Autopilot User Guide*, <https://docs.px4.io/>, 2024.
- [2] Dronecode Foundation, *MAVLink Developer Guide*, <https://mavlink.io/en/>, 2024.
- [3] ArduPilot / Dronecode, *pymavlink: Python MAVLink implementation*, <https://github.com/ArduPilot/pymavlink>, 2024.
- [4] Open Robotics, *Gazebo Classic Simulation Documentation*, <https://classic.gazebosim.org/>, 2023.
- [5] Dronecode Foundation, *QGroundControl User Guide*, <https://docs.qgroundcontrol.com/>, 2024.
- [6] Open Robotics, *ROS 2 Documentation*, <https://docs.ros.org/>, 2024.
- [7] eProsima, *Micro XRCE-DDS Documentation*, <https://micro-xrce-dds.docs.eprosima.com/>, 2024.
- [8] eProsima, *Fast DDS Documentation*, <https://fast-dds.docs.eprosima.com/>, 2024.
- [9] Dronecode Foundation, *MAVSDK Documentation*, <https://mavsdk.mavlink.io/>, 2024.
- [10] V. Ermakov and contributors, *mavros ROS package*, <https://github.com/mavlink/mavros>, 2024.
- [11] S. Ramírez, *FastAPI Documentation*, <https://fastapi.tiangolo.com/>, 2024.
- [12] Encode, *Uvicorn ASGI server*, <https://www.uvicorn.org/>, 2024.
- [13] R. Cabello and contributors, *three.js Documentation*, <https://threejs.org/docs/>, 2024.
- [14] PyTorch Team, *PyTorch Documentation*, <https://pytorch.org/docs/stable/index.html>, 2024.
- [15] F. Pedregosa et al., *scikit-learn User Guide*, <https://scikit-learn.org/stable/>, 2024.
- [16] G. Ke et al., “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” *NeurIPS*, 2017.
- [17] The pandas Development Team, *pandas Documentation*, <https://pandas.pydata.org/docs/>, 2024.

- [18] NumPy Developers, *NumPy Documentation*, <https://numpy.org/doc/>, 2024.
- [19] P. Biondi and the Scapy community, *Scapy Documentation*, <https://scapy.readthedocs.io/>, 2024.
- [20] The Tcpdump Group, *tcpdump / libpcap*, <https://www.tcpdump.org/>, 2024.
- [21] Canonical Ltd., *Ubuntu Documentation*, <https://ubuntu.com/server/docs>, 2024.
- [22] Kitware, *CMake Documentation*, <https://cmake.org/documentation/>, 2024.
- [23] M. Grieves, “Digital Twin: Manufacturing Excellence through Virtual Factory Replication,” 2012.
- [24] R. Altawy and A. M. Youssef, “Security of Unmanned Aerial Vehicles: A Survey,” *ACM Computing Surveys*, 2016.
- [25] Joblib developers, *joblib Documentation*, <https://joblib.readthedocs.io/>, 2024.
- [26] Paramiko Project, *Paramiko Documentation*, <https://www.paramiko.org/>, 2024.

Linux Command Reference

.1 Filesystem

```
pwd; ls -la; cd DIR; mkdir -p DIR; cp -r A B; mv A B; rm -rf DIR  
cat FILE; less FILE; grep -R pattern DIR; df -h; du -sh DIR
```

.2 Packages

```
sudo apt update  
sudo apt install -y PKG  
sudo apt remove PKG
```

.3 Networking

```
ip -br addr  
ip route get 192.168.123.130  
ping -c 3 192.168.123.130  
ssh danish@192.168.123.130  
scp FILE danish@192.168.123.130:~/  
sudo ss -ulnp | grep -E '14550|18570|22'
```

.4 Processes

```
ps aux | grep -E 'px4|gzserver|gzclient'  
kill PID  
htop # if installed
```

ROS 2 Command Reference (Optional Path)

Operational DT/IDS does not require ROS 2. These commands support the educational chapters.

```
source /opt/ros/$ROS_DISTRO/setup.bash
echo $ROS_DOMAIN_ID
ros2 topic list
ros2 topic echo /TOPIC --once
ros2 node list
ros2 interface list
ros2 doctor
ros2 launch PKG FILE.py
ros2 run demo_nodes_cpp talker
ros2 run demo_nodes_cpp listener
```

.5 DDS notes

Hosts must share `ROS_DOMAIN_ID` and be multicast-reachable [6, 8]. Micro XRCE-DDS bridges PX4 to ROS 2 when that optional path is enabled [7].

PX4, Gazebo, DDS, and MAVLink References

.6 PX4 SITL

```
cd ~/PX4-Autopilot
bash ./Tools/setup/ubuntu.sh
make px4_sitl gazebo-classic
# PX4 shell examples
commander takeoff
commander land
```

.7 Gazebo Classic

```
export DISPLAY=:0
gzserver --verbose &
gzclient
```

.8 Remote lab scripts

```
ssh danish@192.168.123.130 \
'cd ~/uav_cyber_testbed && ./scripts/start_sitl_baseline.sh'
```

.9 MAVLink / pymavlink on DT host

```
.venv/bin/python -c "from pymavlink import mavutil; print('pymavlink ok')"
```

```
# Gateway listen port
# 127.0.0.1:19550 -> UAV 192.168.123.130:14550
```

.10 Ports quick sheet

22/tcp SSH; 8000/tcp dashboard; 14550/udp telemetry; 18570/udp GCS API; 19550/udp local proactive gateway; 14540/udp RX; 14580/udp offboard; 8888 XRCE.

Folders, Environment, Topology, FAQ, Glossary

.11 Folder structure

```
/Users/danishvasan/Downloads/uav-cyber-ml/  
attacks/ # suite.py, benign.py  
dashboard/ # server.py, static/app.js  
datasets/ # runs/, DATA_DICTIONARY.md  
ids/ # cnn_*, train, live_bridge, mav_gateway, artifacts/  
recorders/ # physical/network/twin bridges  
docs/technical_manual/  
orchestrator.py mav_common.py ssh_control.py config.py  
requirements.txt run_dashboard.sh scripts/
```

.12 Environment variables

Variable	Meaning
UAV_HOST	UAV IP (default 192.168.123.130)
UAV_SSH_USER	SSH user (default danish)
MAV_GATEWAY	1 enable / 0 bypass proactive GW
MAV_GATEWAY_PORT	default 19550
NET_IFACE	tcpdump interface override
DEFENSE_MODE	proactive hybrid reactive ...
IDS_PRIMARY_MODEL	cnn1d fusion
GPS_SPOOF_DRIFT	deg/s spoof severity
FLIGHT_PROFILE	mission profile key
ATTACK_AFTER_WP	first eligible gate index
ATTACK_GATE_FRACTION	fraction of mid WPs as gates

.13 Port assignments

22/tcp SSH; 8000/tcp dashboard; 14550/udp telemetry; 18570/udp API; 19550/udp gateway (localhost); 14540 RX; 14580 offboard.

.14 Network topology

DT host (UI+IDS+gateway) — LAN 192.168.123.0/24 — UAV PC (PX4+Gazebo @ 192.168.123.130).
Dashboard and gateway bind localhost on the DT host.

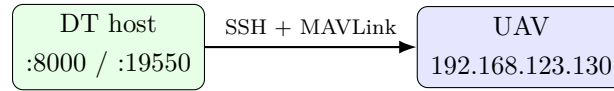


Figure 1: Appendix topology sketch.

.15 FAQ

DT not moving? Start sim; close QGC; check `/api/pt`.

Defence disabled? Train CNN; ensure artefacts on disk; enable Defence.

SSH timeout? Join 192.168.123.0/24; install SSH keys.

Empty network CSV? Fix tcpdump privileges / `NET_IFACE`.

Proactive blocks stay 0? Mode proactive/hybrid, Defence ON, model loaded.

ModuleNotFoundError? Use `.venv/bin/python / run_dashboard.sh`.

.16 Glossary

PT Physical Twin; **DT** Digital Twin; **IDS** Intrusion Detection System; **SITL** Software-in-the-Loop; **GCS** Ground Control Station; **EKF** Extended Kalman Filter; **DDS** Data Distribution Service; **OFFBOARD** PX4 mode accepting external setpoints; **TinyMAV** lab 1D-CNN on MAVLink network windows; **Gateway** local UDP filter before PX4; **Sysid** MAVLink system id (attacker 250, pilot 252, ...).